

Deep learning

episode 2, 2023

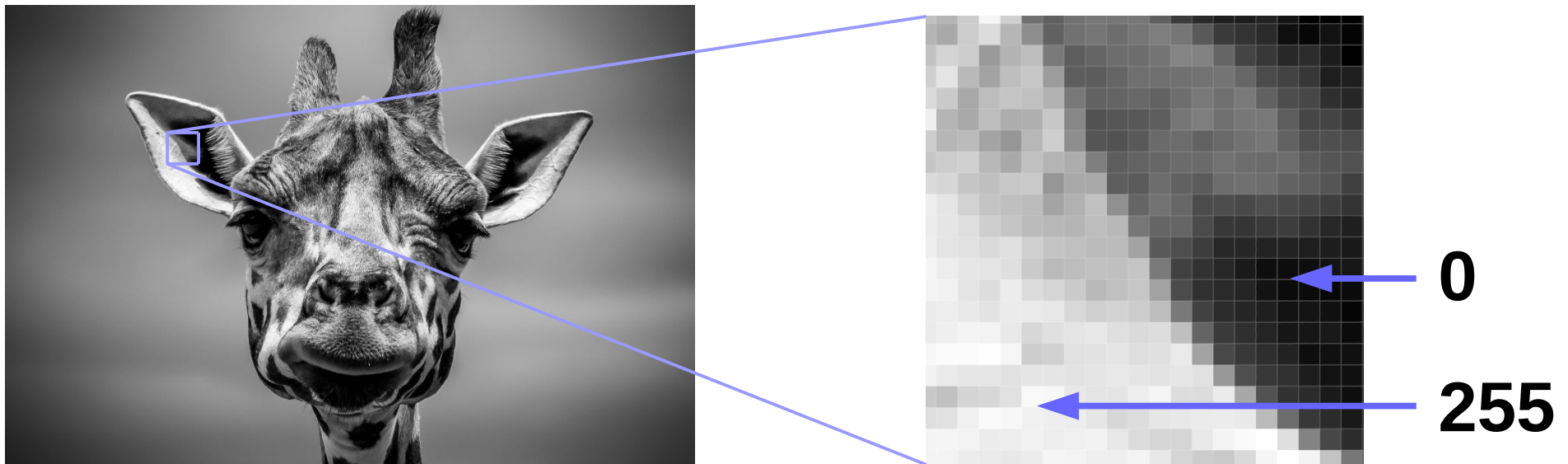
Convolutional Neural Networks

Yandex Research



Images

- Grayscale image is a matrix of pixels [H x W]
 - Pixels = **p**icture **e**lements
- Each pixel stores number [0,255] for brightness



Images

- RGB image is a 3d array $[H \times W \times 3]$ or $[3 \times H \times W]$
 - Each pixel stores **R**ed, **G**reen & **B**lue color values $[0, 255]$

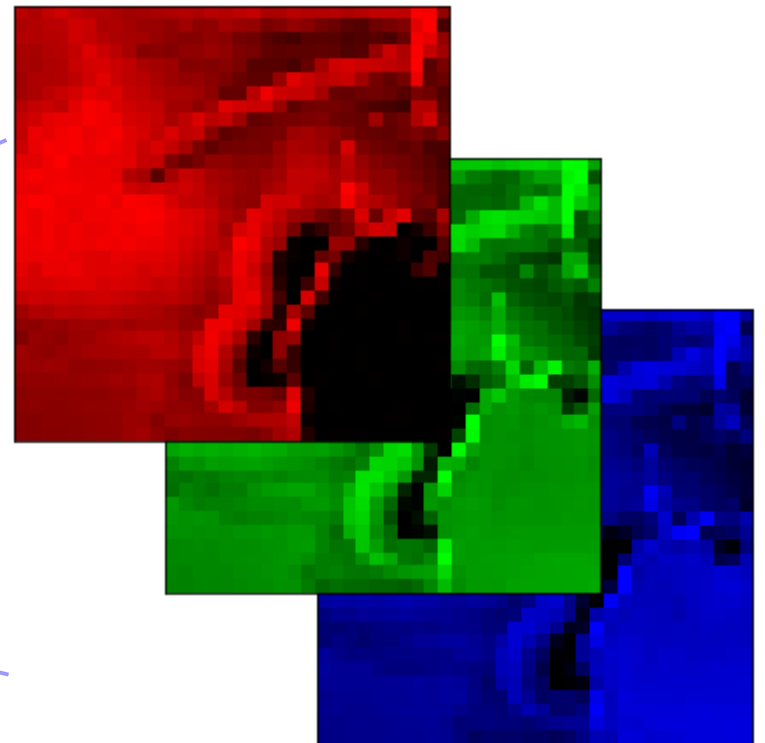
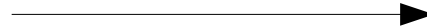
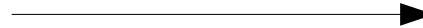


Image recognition



“Dog”

Image recognition



“Gray wall”

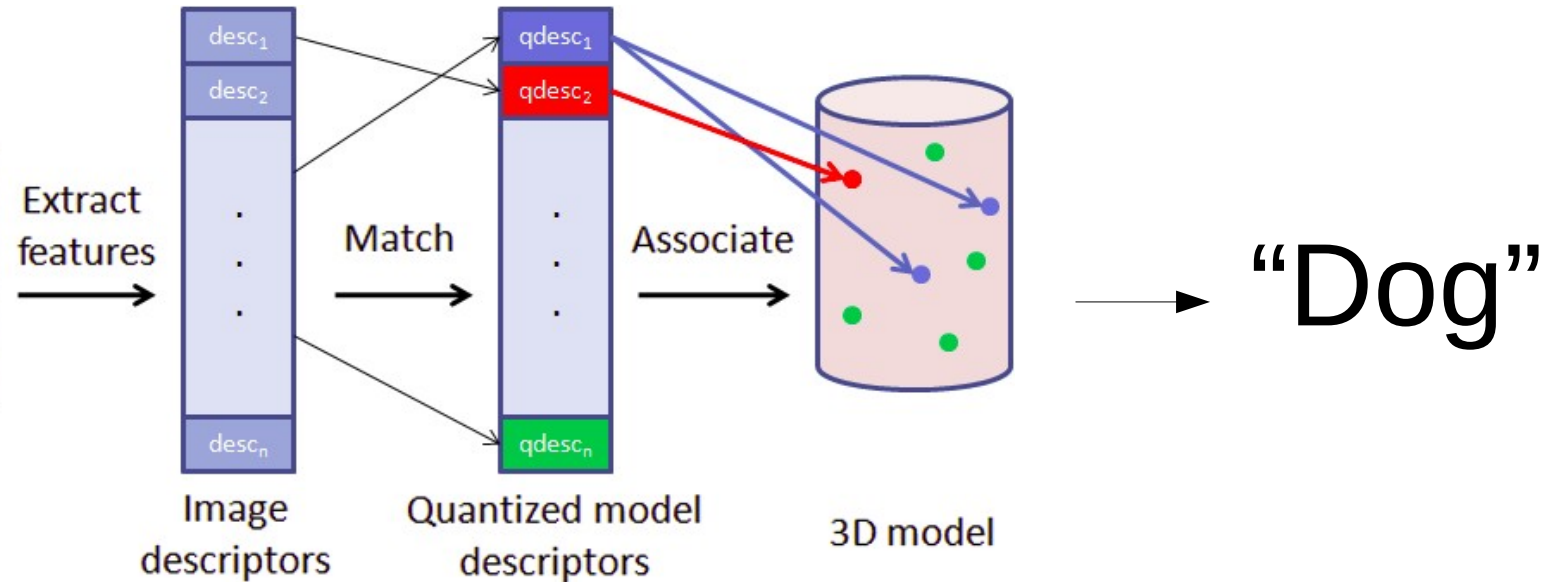
“Dog tongue”

“Dog”

<a particular kind
of dog>

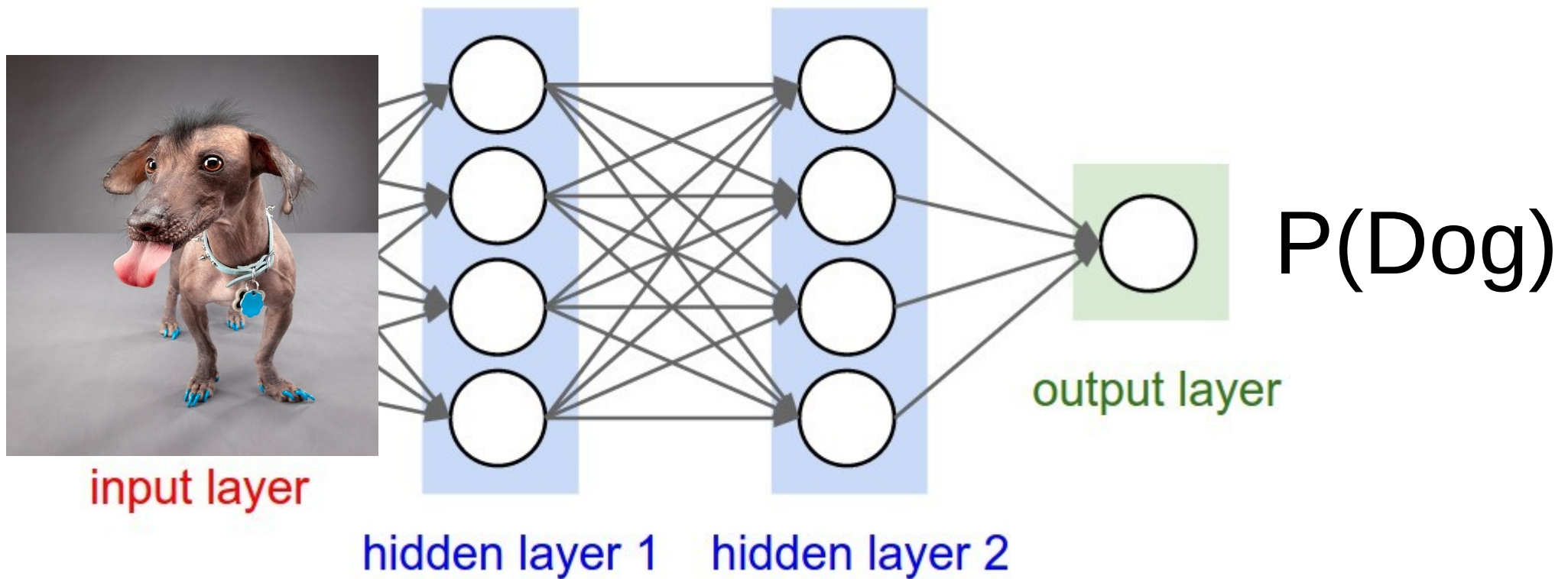
“Animal sadism”

Classical approach

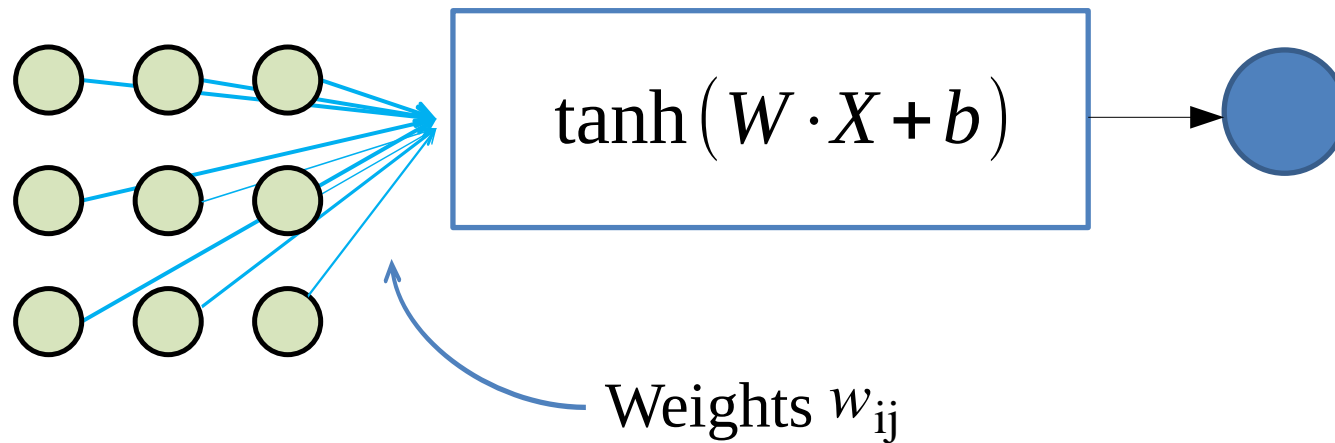


Guess what we're going to do now?

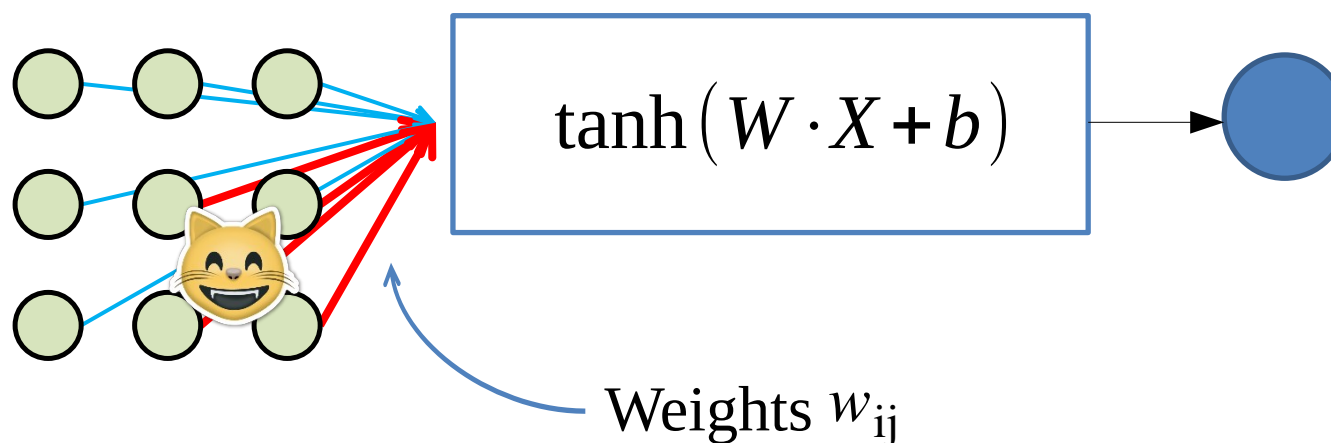
NN approach



Problem with images

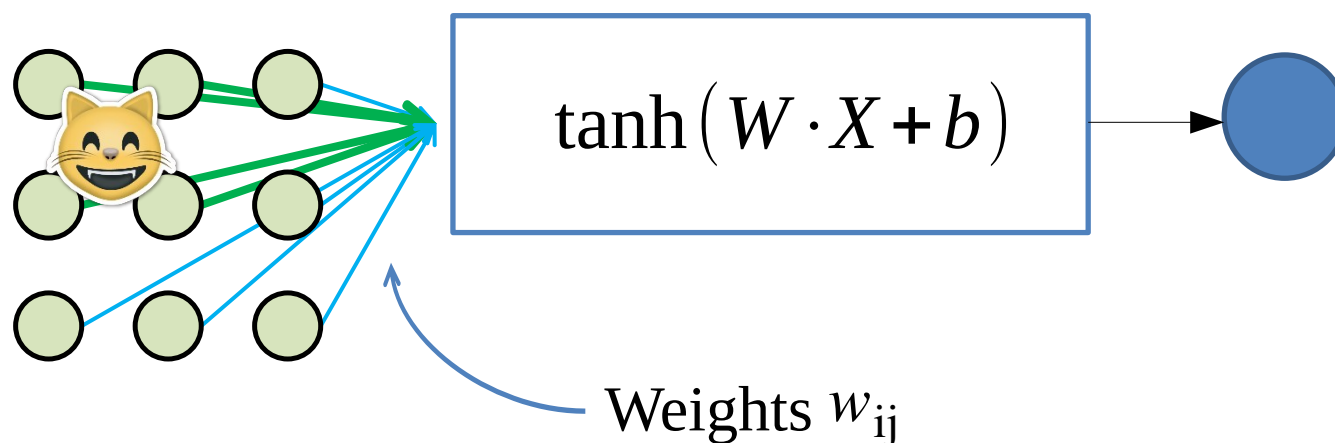


Problem with images



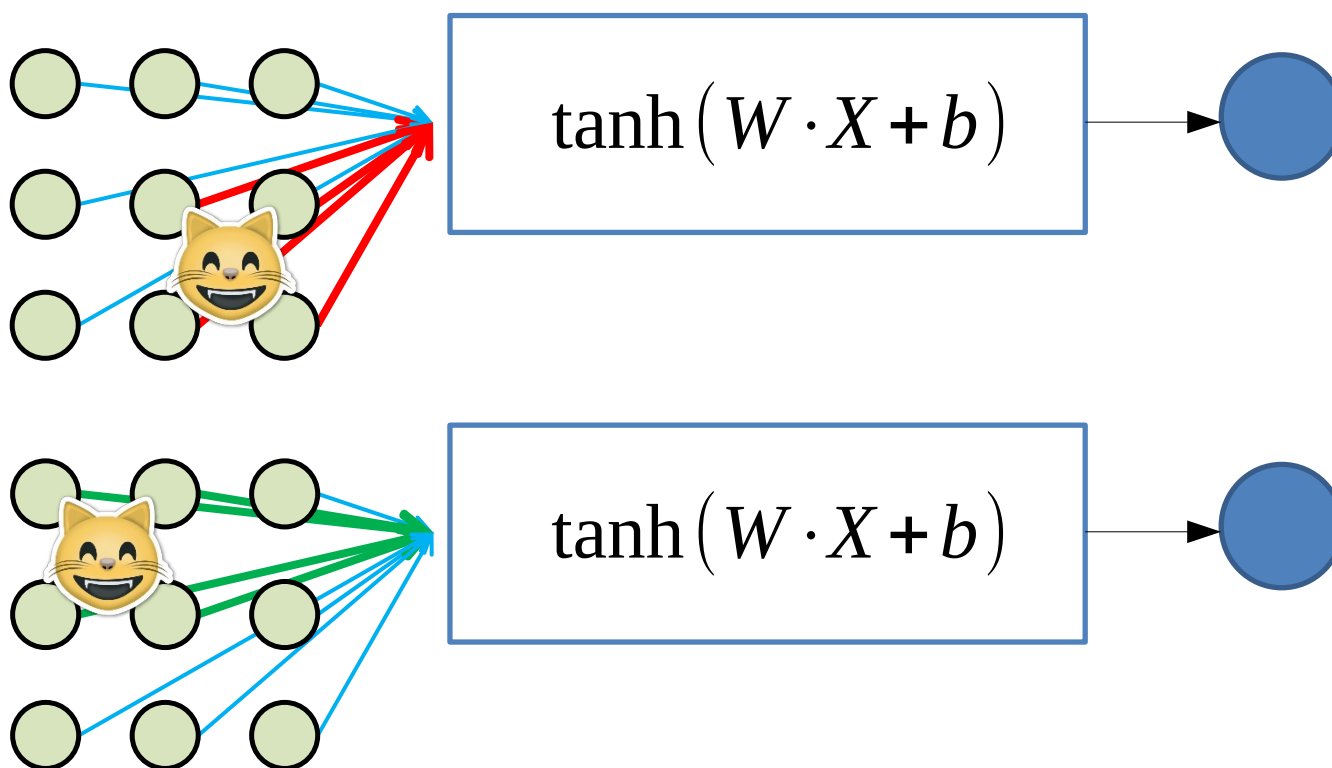
On this object, you will train **red** weights to react on cat face

Problem with images



On this object, you will train **green** weights to react on cat face

Problem with images



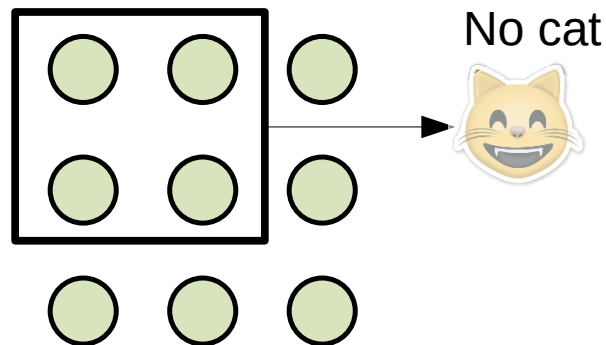
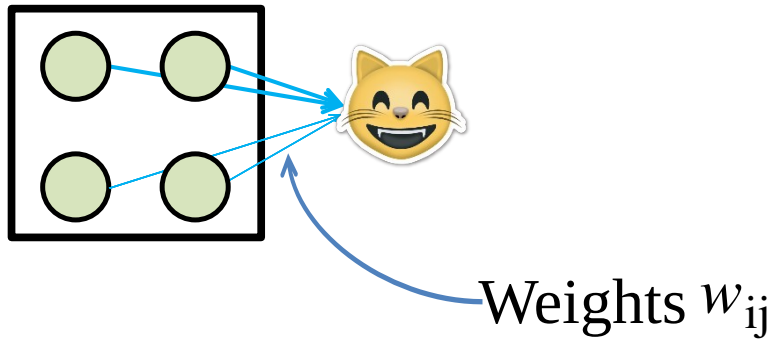
You network will have to learn those two cases separately!
Worst case: one neuron per position.

Problem

Idea: force all these “cat face” features
to use **exactly same weights**,
shifting weight matrix each time.

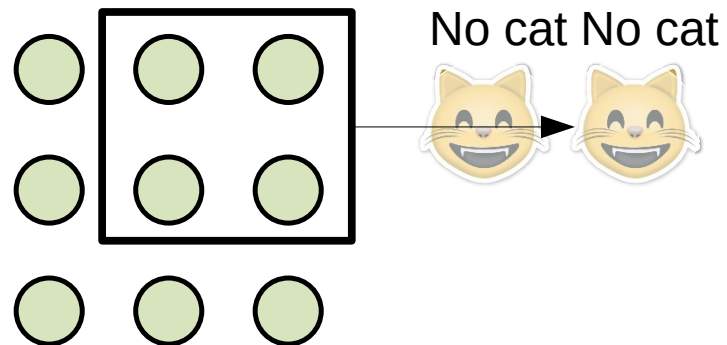
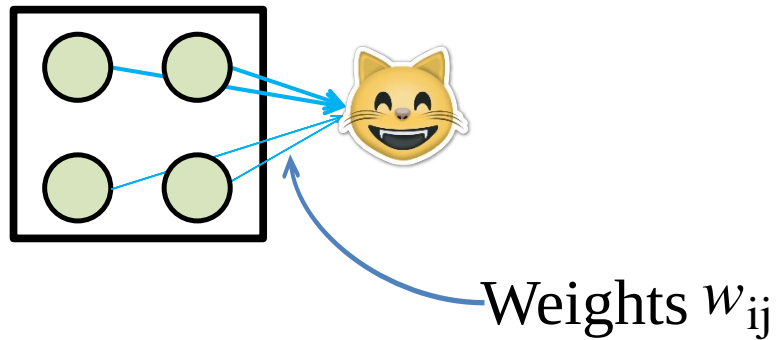
Same features for each spot

Portable cat detector pro!



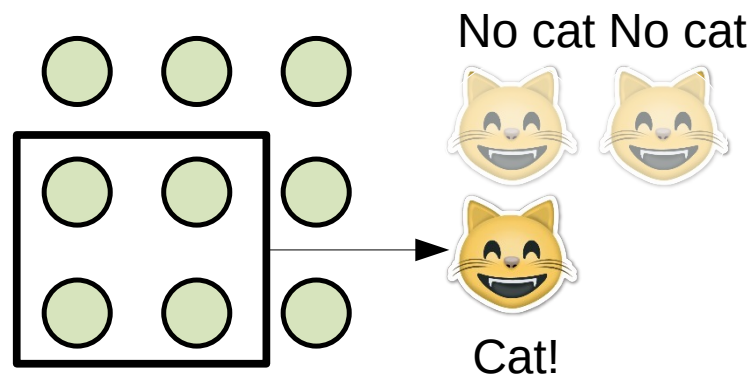
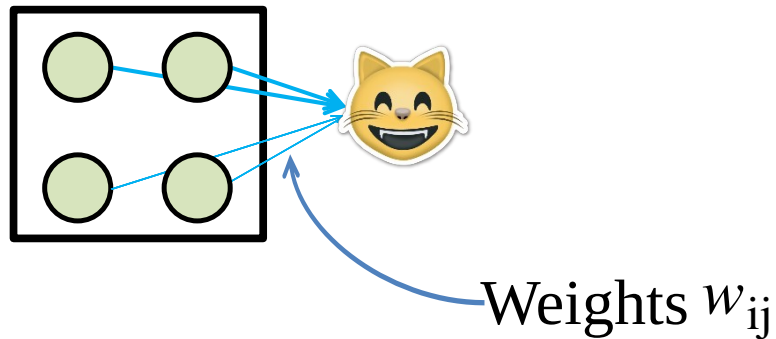
Same features for each spot

Portable cat detector pro!



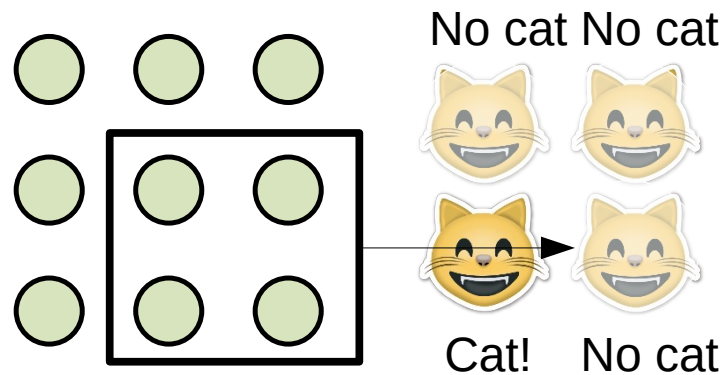
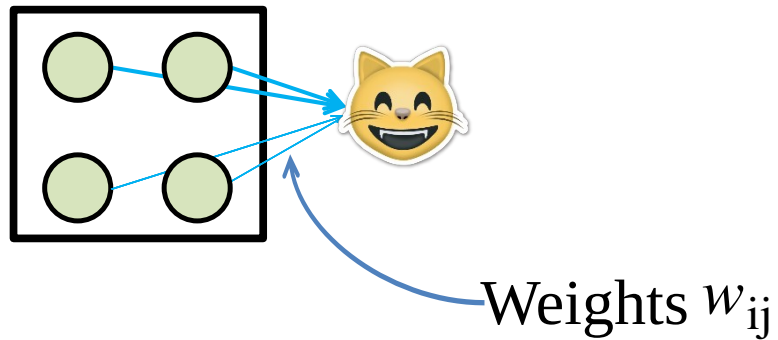
Same features for each spot

Portable cat detector pro!



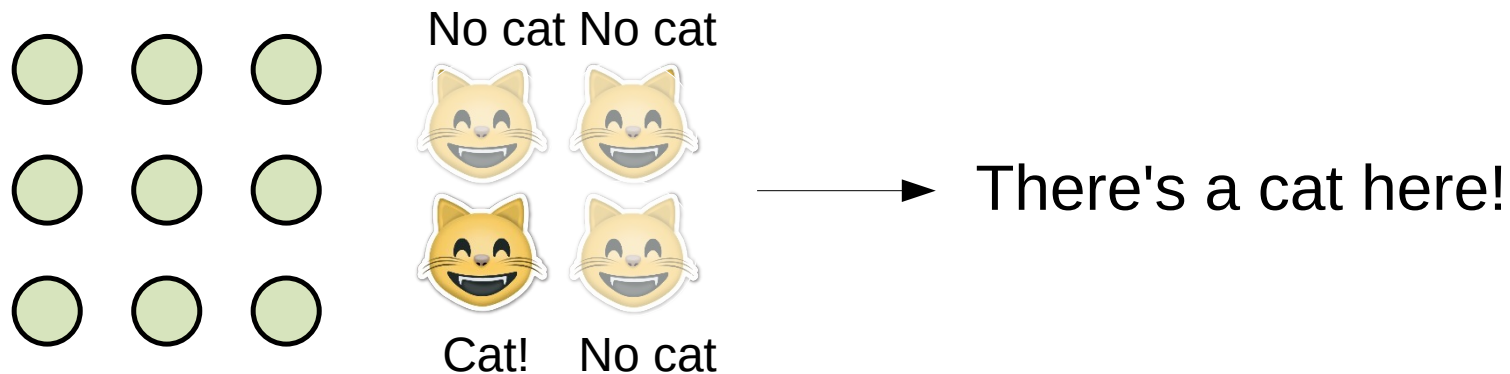
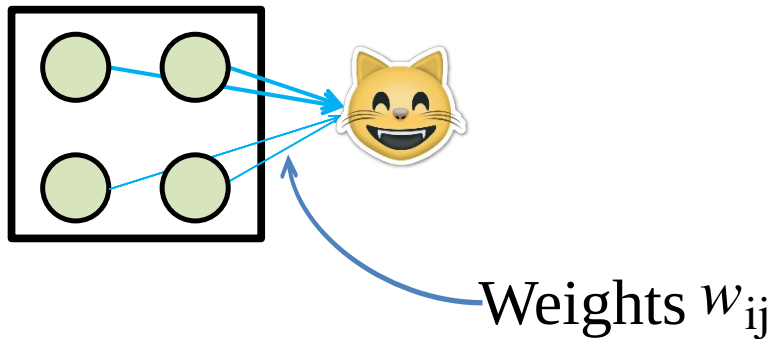
Same features for each spot

Portable cat detector pro!



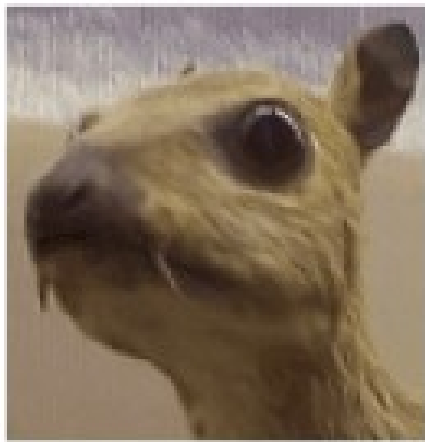
Same features for each spot

Portable cat detector pro!



Convolution

Input image



Convolution
Kernel

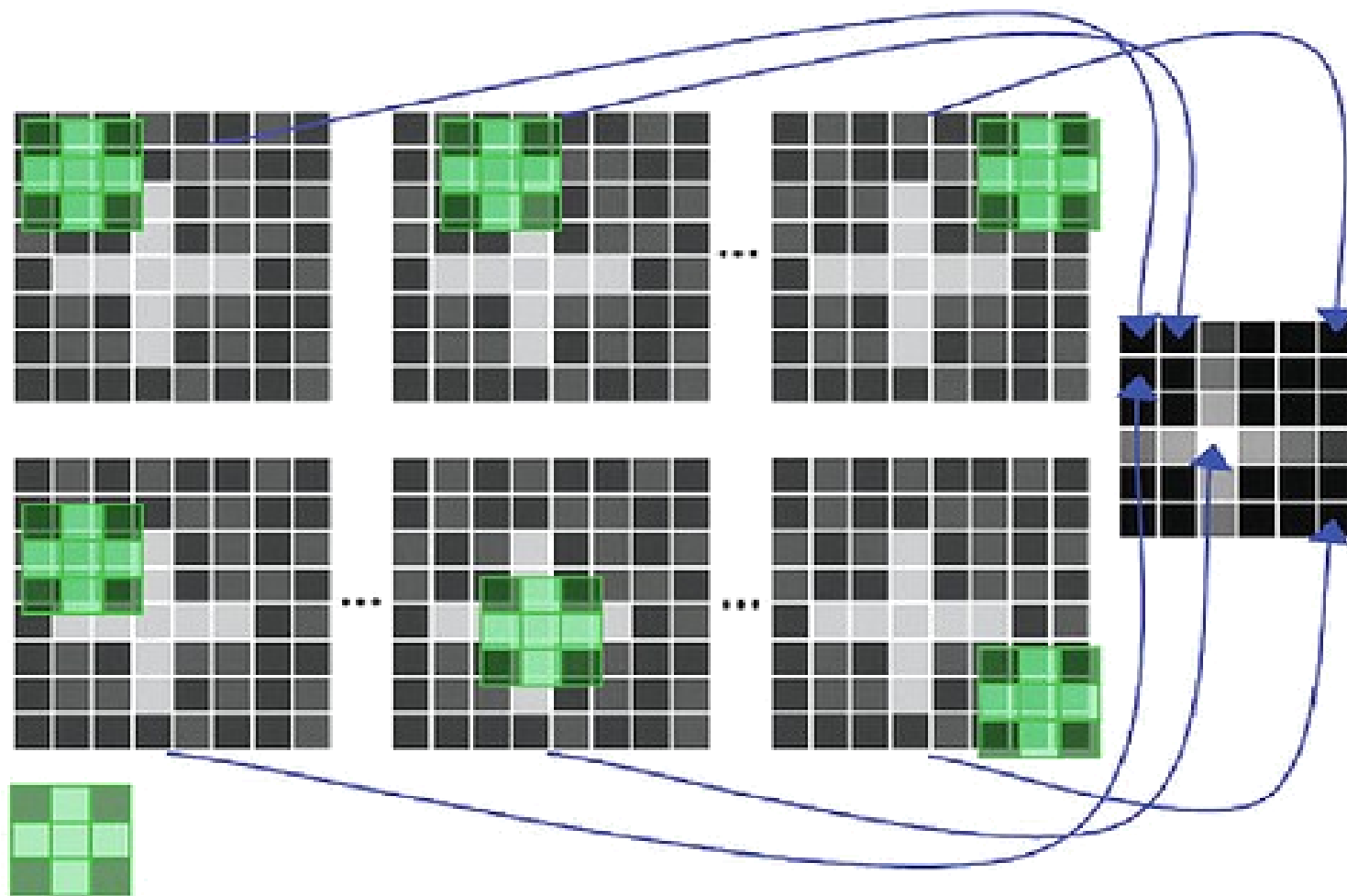
$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map



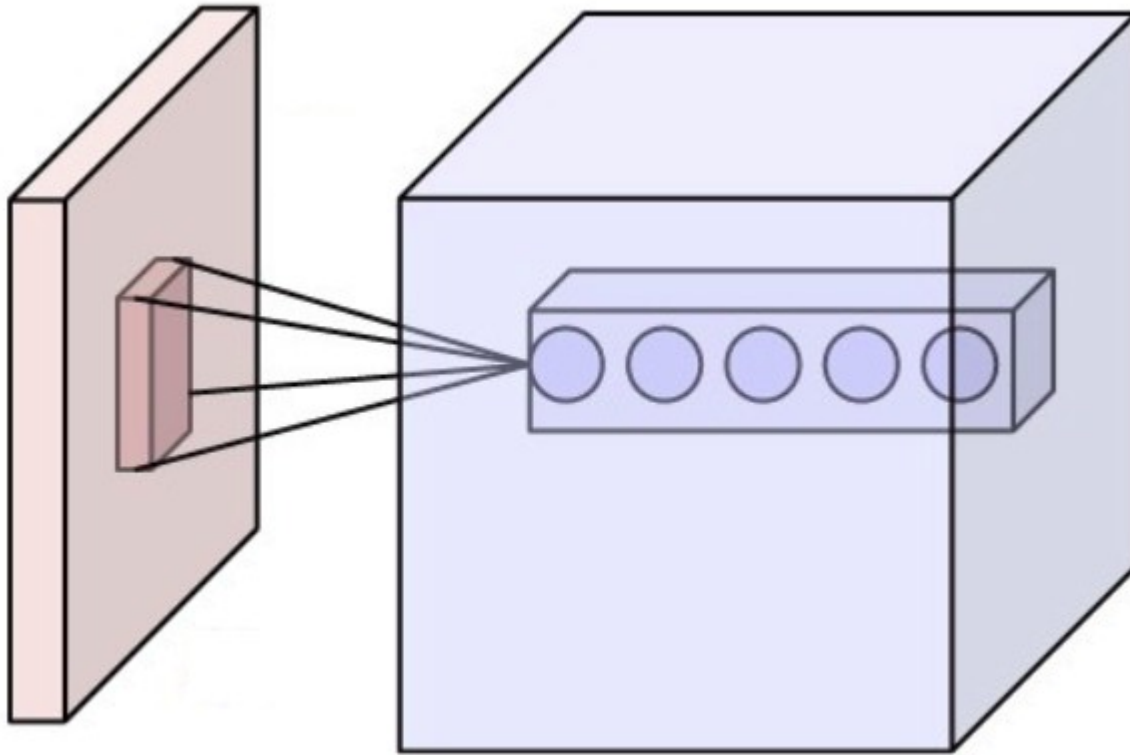
Intuition: how **edge-like** is this square?

Convolution



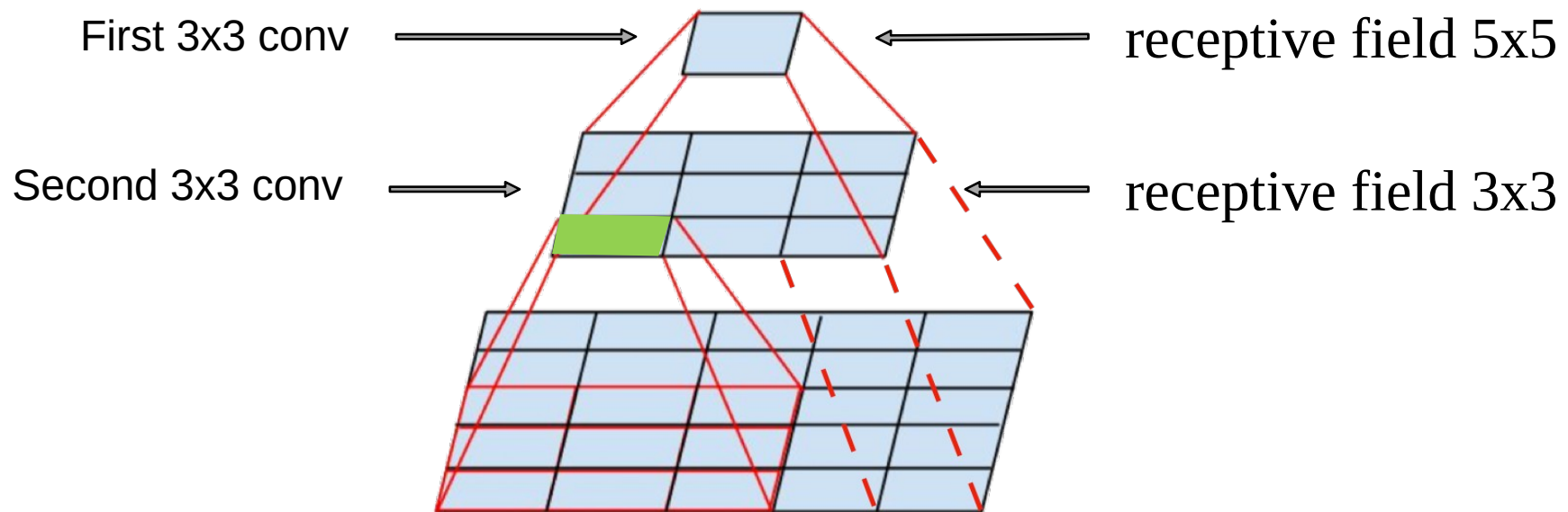
apply same filter to all patches

Convolution



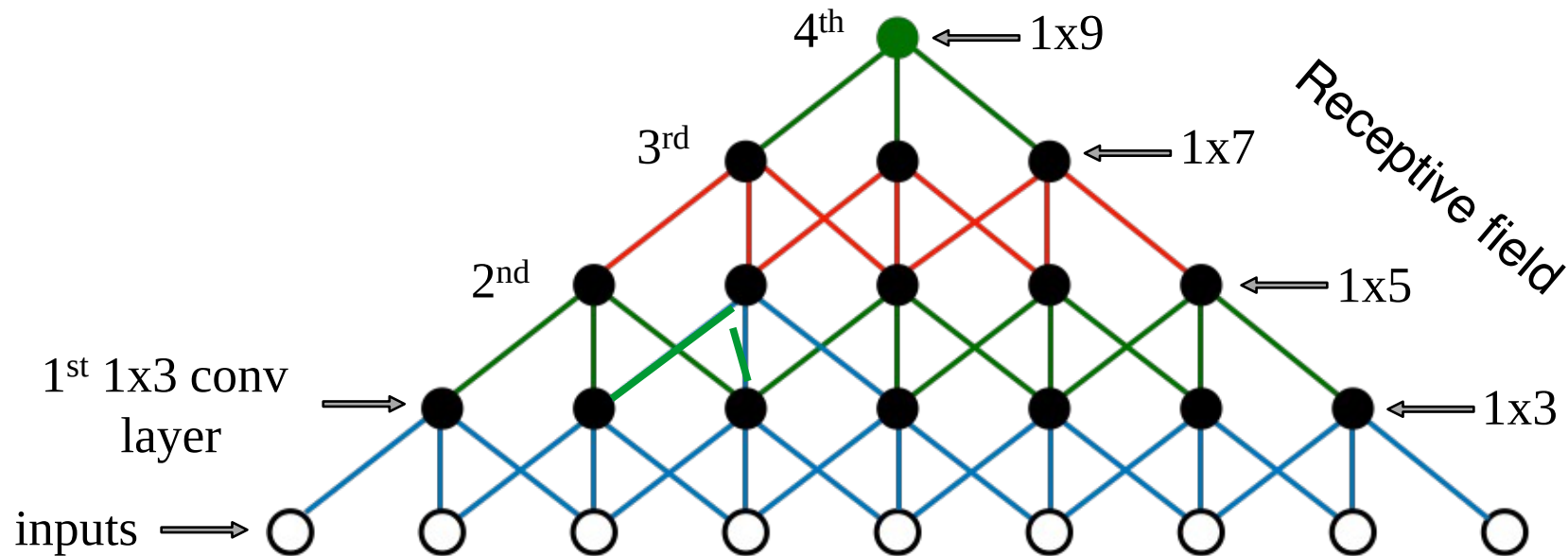
Intuition: how cat-like is this square?

Receptive field



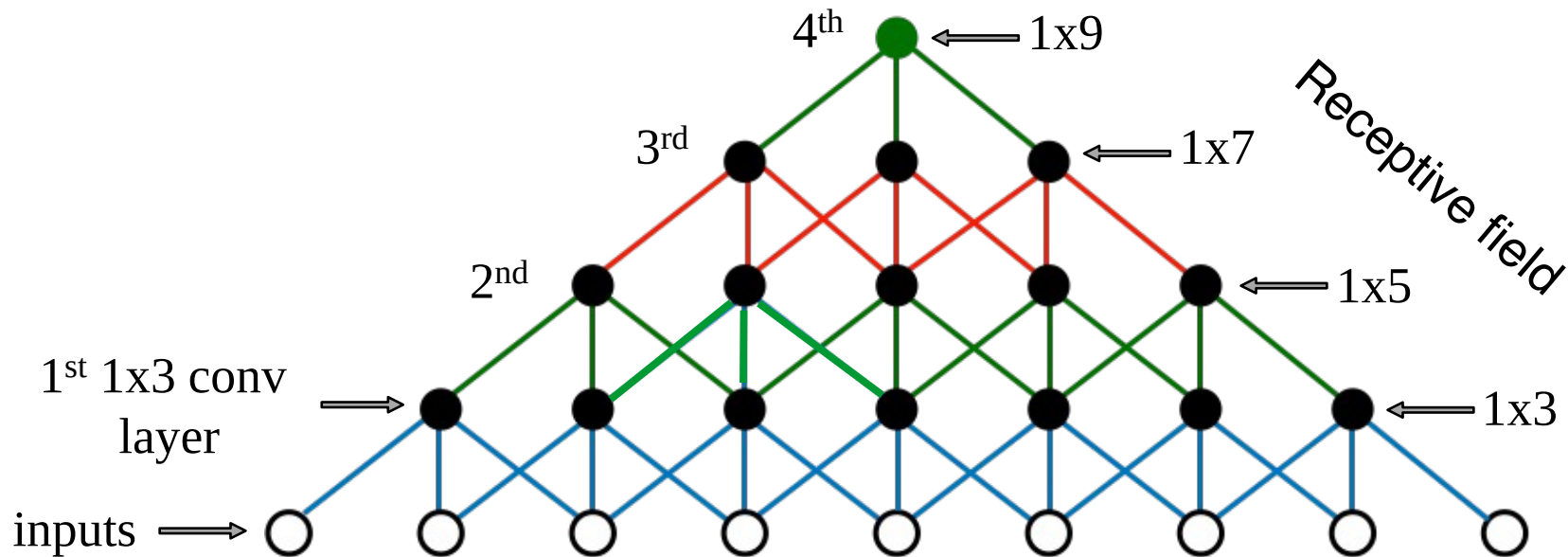
We can recognize larger objects by stacking several small convolutions!

Receptive field



Q: how many 3x3 convolutions we should use to recognize a 100x100px cat

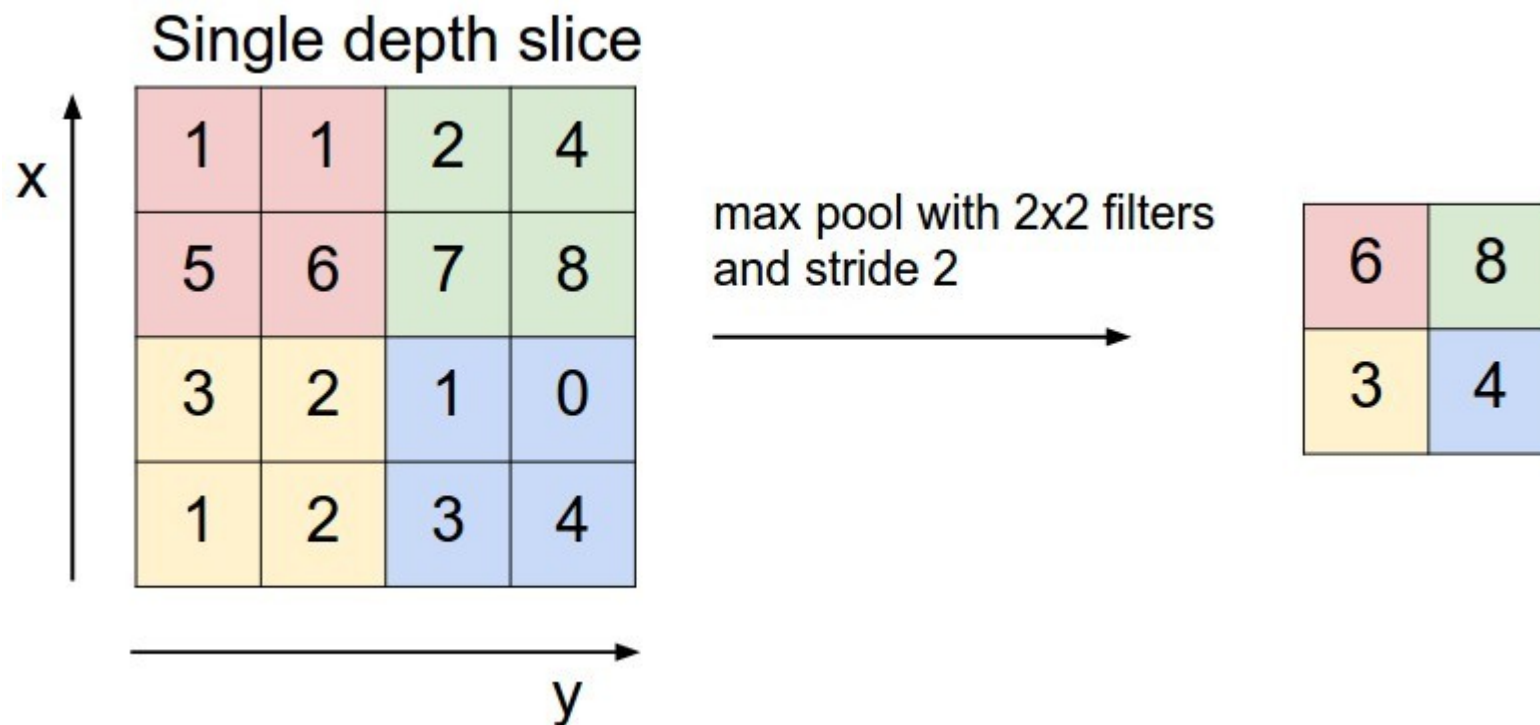
Receptive field



Q: how many 3x3 convolutions we should use to recognize a 100x100px cat

A: around 50... we need to increase receptive field faster!

Pooling



Intuition: What is the max cat-likelihood over this area?

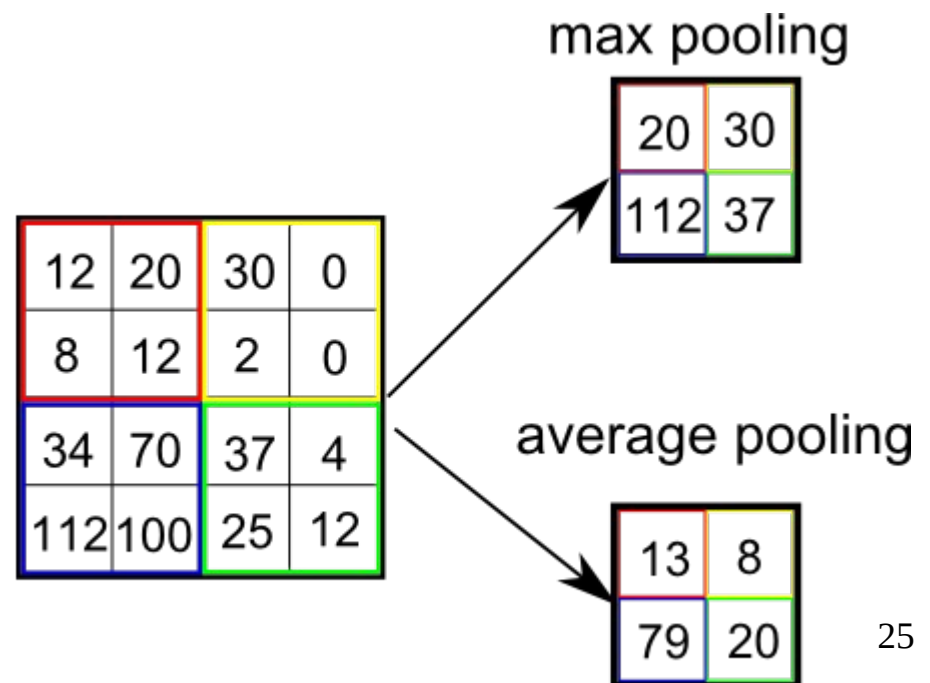
Pooling

Motivation:

- Reduce layer size by a factor
- Make NN less sensitive to small image shifts

Popular types:

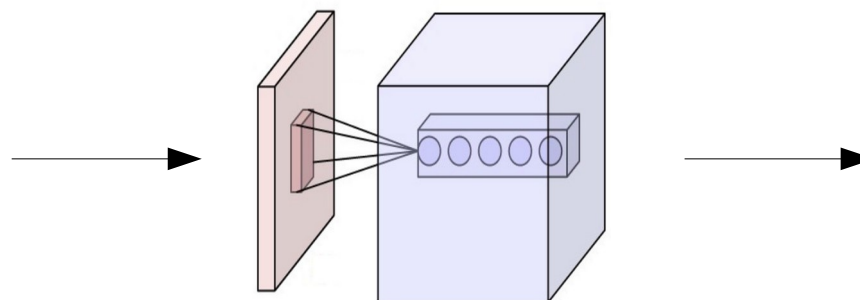
- Max
- Mean(average)



Convolution



Image : 3 (RGB) x 100 px x 100 px

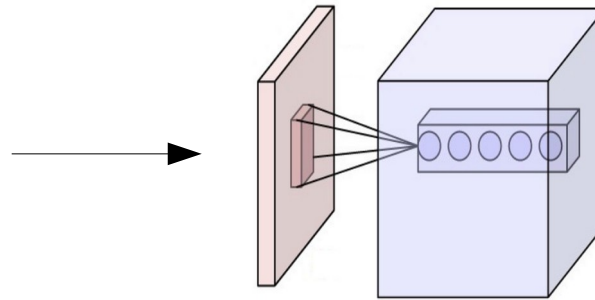


Filters: 100x(3x5x5)

Convolution



Image : 3 (RGB) x 100 px x 100 px



Filters: 100x(3x5x5)

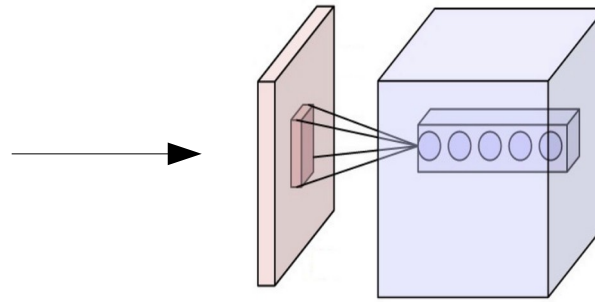
100x96x96
~10⁶

Somewhat too many!

Convolution



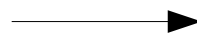
Image : 3 (RGB) x 100 px x 100 px



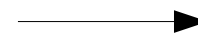
Filters: 100x(3x5x5)

100x96x96
~10⁶

100x96x96



pool
3x4

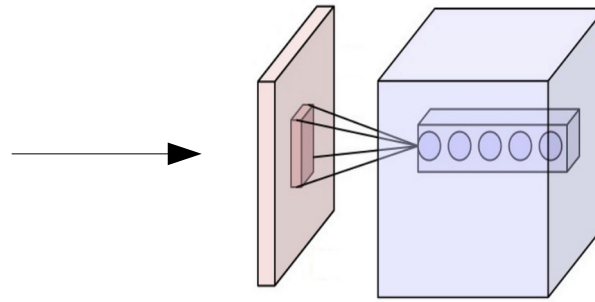


???

Convolution



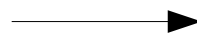
Image : 3 (RGB) x 100 px x 100 px



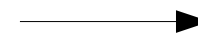
Filters: 100x(3x5x5)

100x96x96
~10⁶

100x96x96



pool
3x4



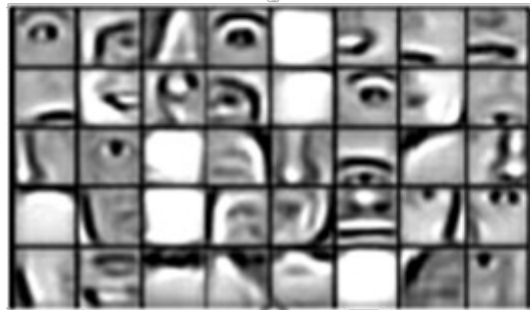
100x32x32

~10⁵

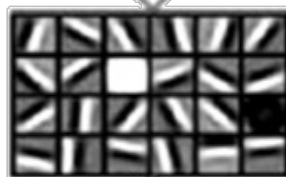


Discrete Choices

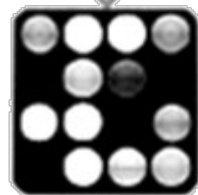
⋮



Layer 2 Features

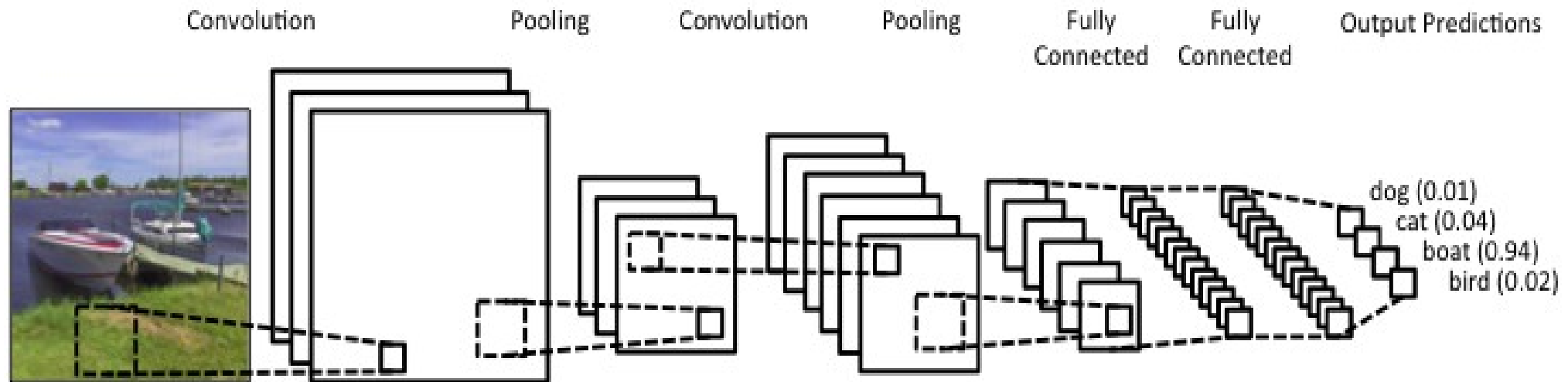


Layer 1 Features

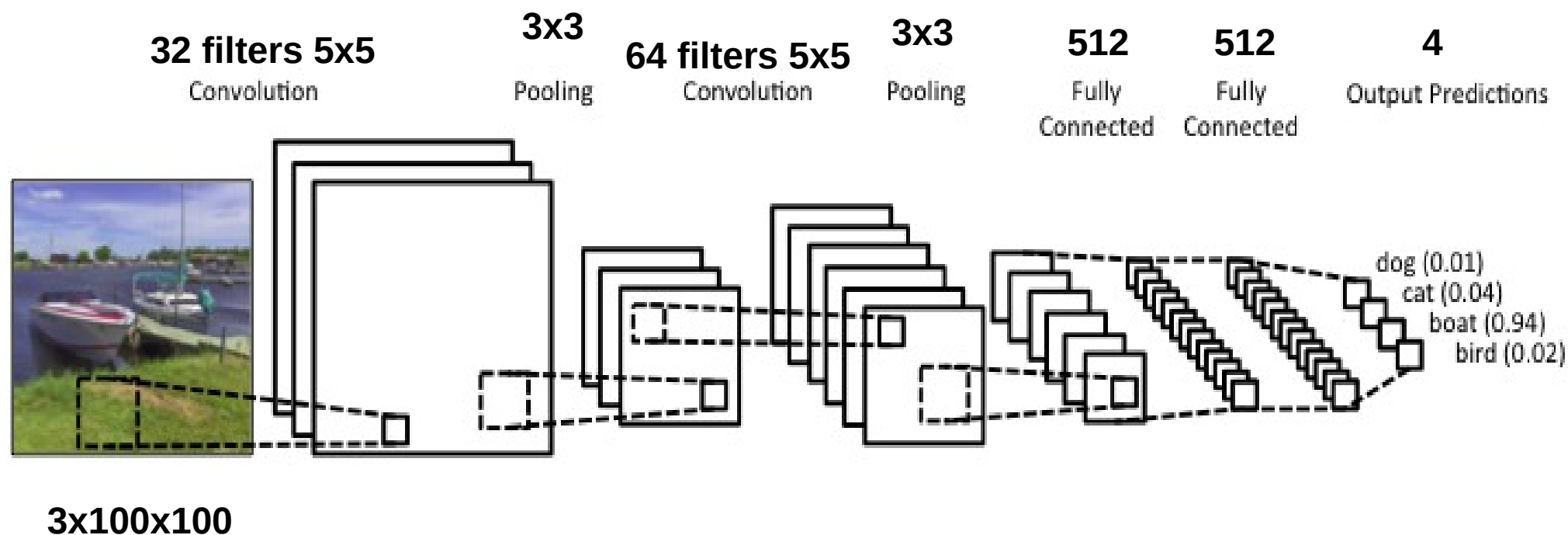


Original Data

Convolutional NNs



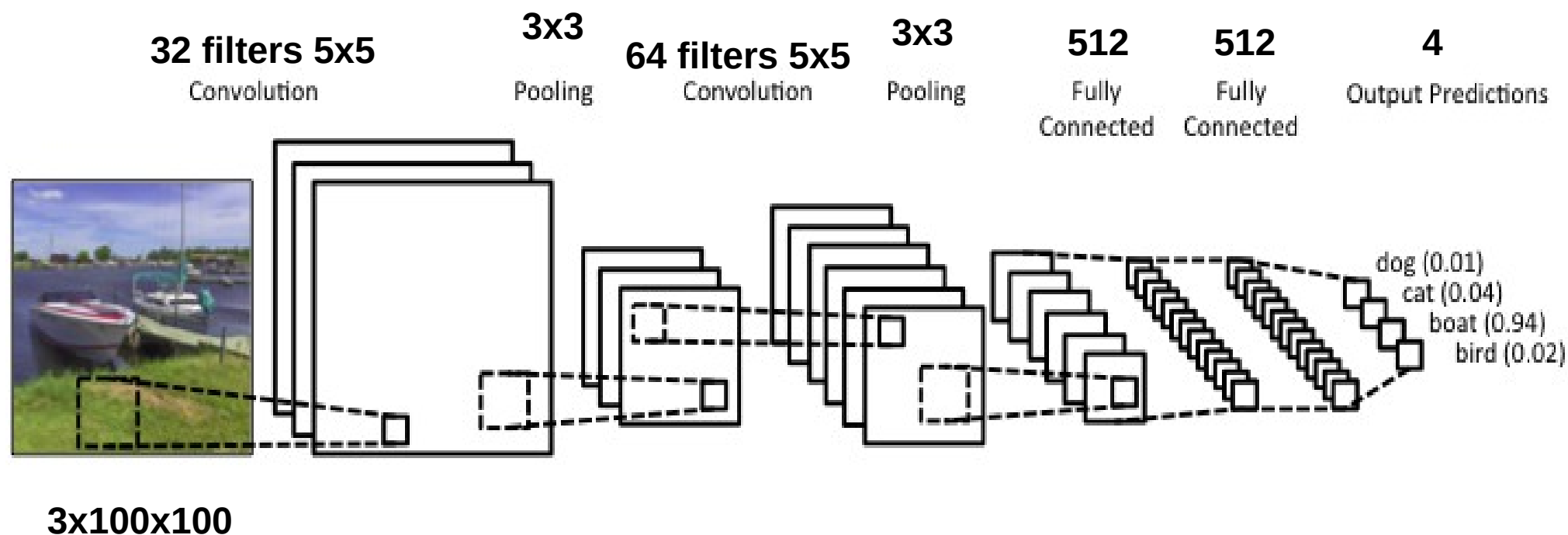
Convolutional NNs



Quiz:

1) What is the output shape **after second pooling**

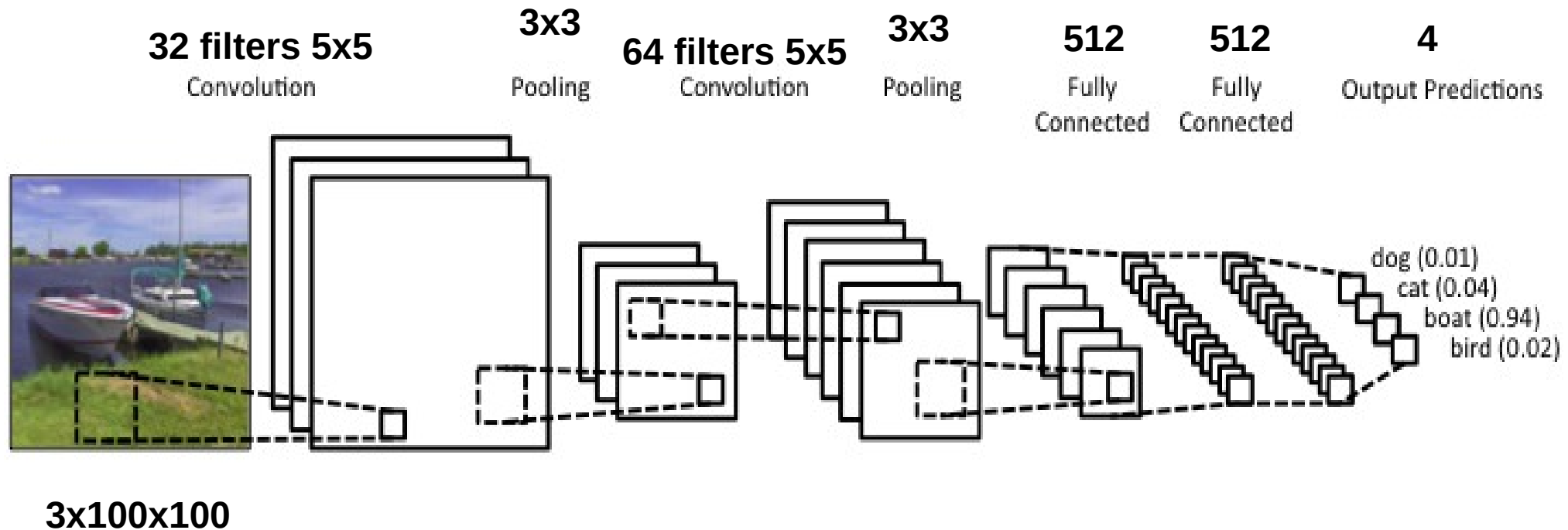
Convolutional NNs



Quiz:

2) How many image pixels does **one cell** after **second convolution** depend on?

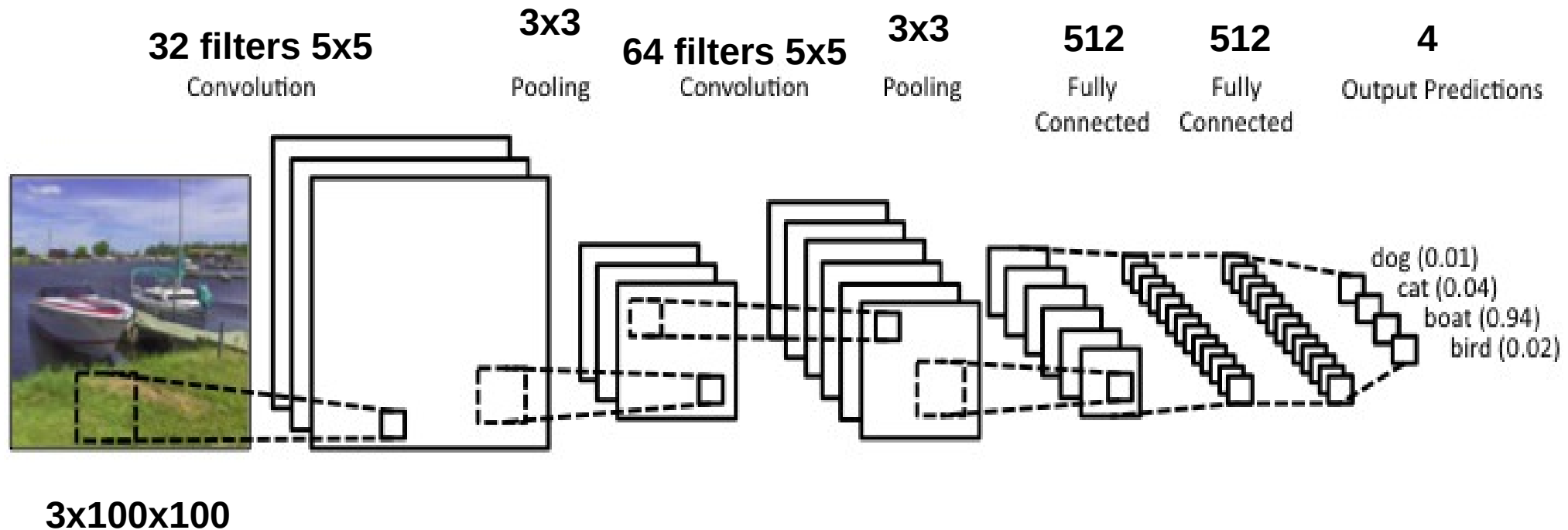
Convolutional NNs



Quiz:

- 3) Which layer is hardest to compute?
- 4) Which layer has most independent parameters?

Convolutional NNs



Quiz:

- 3) Which layer is hardest to compute?: **first conv**
- 4) Which layer has most independent parameters?

first dense

Problem with large networks

What you sign for if you stack 1000 layers:

- `MemoryError(0x...)`
- Gradients can vanish
- Gradients can explode
- Activations can vanish
- Activations can explode
- Overfits like crazy

Recap: Batch normalization

Idea:

- We normalize activation of a hidden layer
(zero mean unit variance)

$$h_i = \frac{h_i - \mu_i}{\sqrt{\sigma_i^2}}$$

i stands for i-th neuron

- Update μ_i, σ_i^2 with moving average while training

$$\mu_i := \alpha \cdot \text{mean}_{\text{batch}} + (1 - \alpha) \cdot \mu_i$$

$$\sigma_i^2 := \alpha \cdot \text{variance}_{\text{batch}} + (1 - \alpha) \cdot \sigma_i^2$$

Architectures: residual network

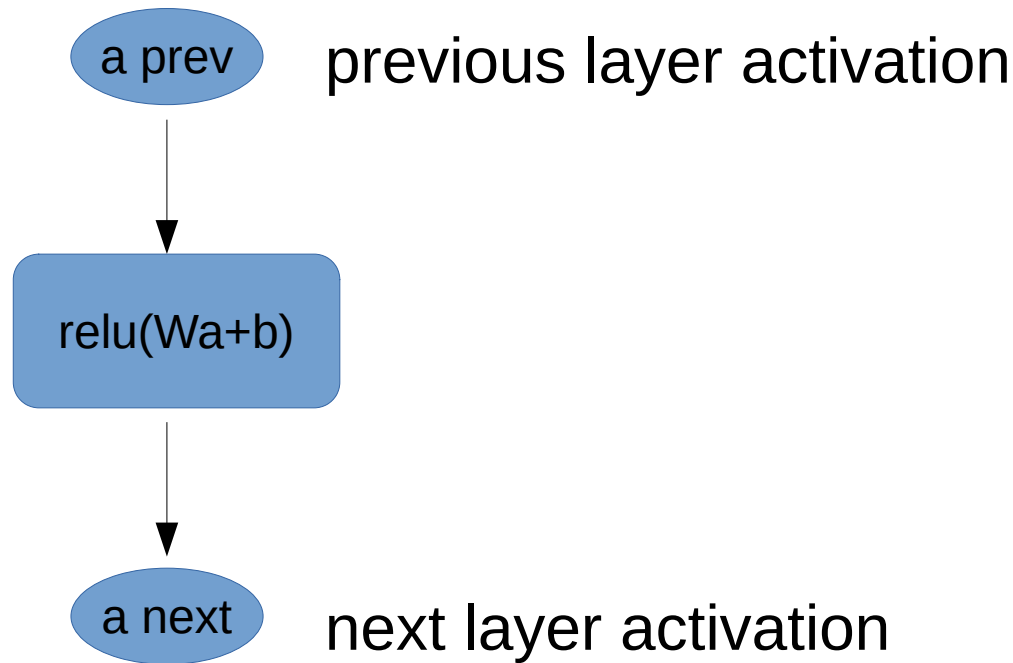
Problem: very deep networks are hard to train

Gradients w.r.t. first layers are volative

Architectures: residual network

Idea: let's create a shortcut for gradients

Normal layer

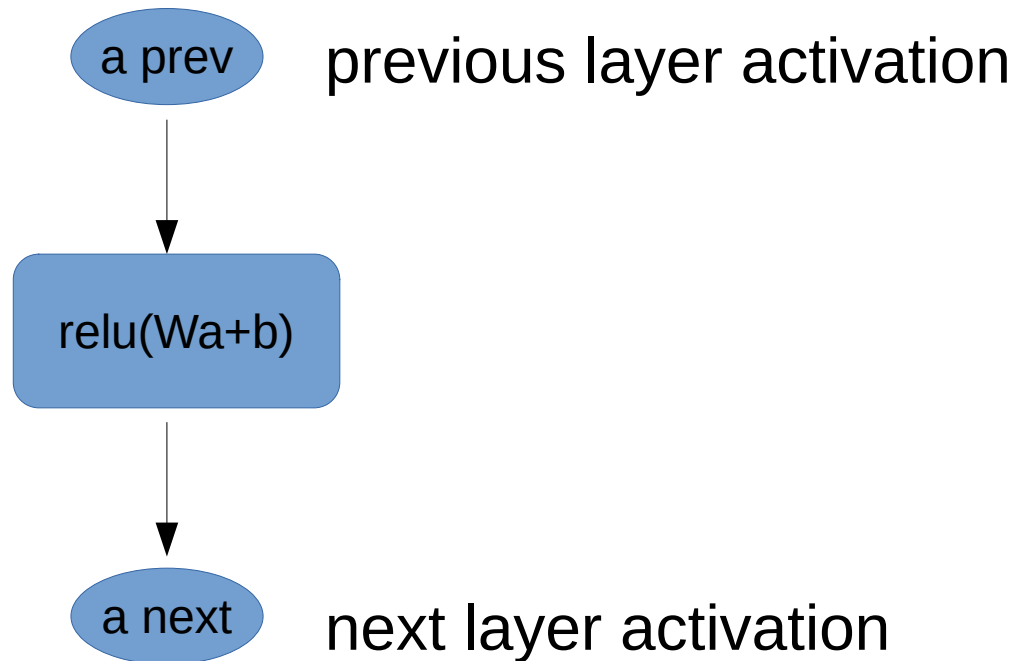


$$f_{w,b}(x) = \text{relu}(W \cdot a + b)$$

Architectures: residual network

Idea: let's create a shortcut for gradients

Normal layer



$$f_{w,b}(x) = \text{relu}(W \cdot a + b)$$

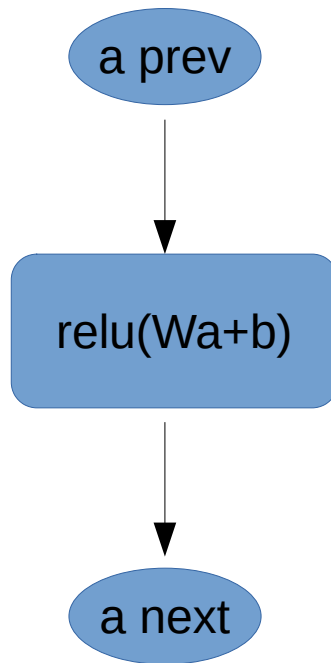
$$\nabla f_{w,b}(x) = \underline{\nabla \text{relu}(W \cdot a + b)}$$

Gradients can vanish if relu < 0

Architectures: residual network

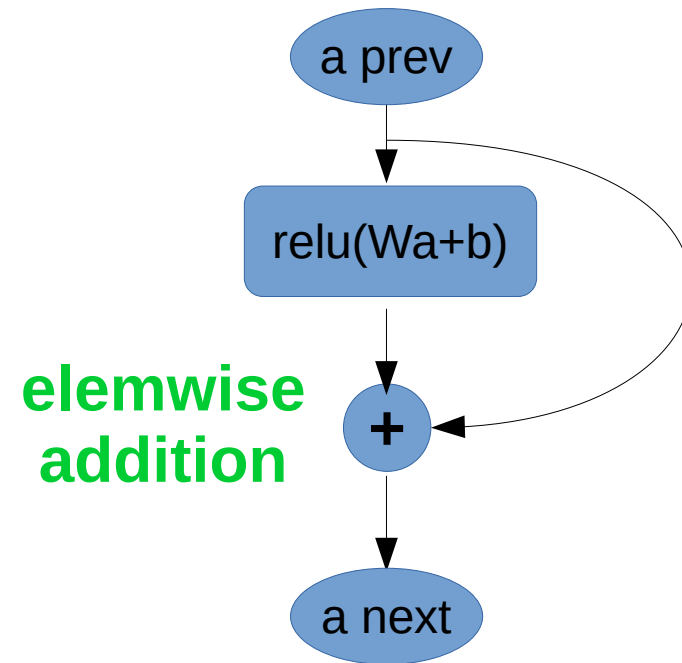
Idea: let's create a shortcut for gradients

Normal layer



$$f_{w,b}(x) = \text{relu}(W \cdot a + b)$$

Residual layer

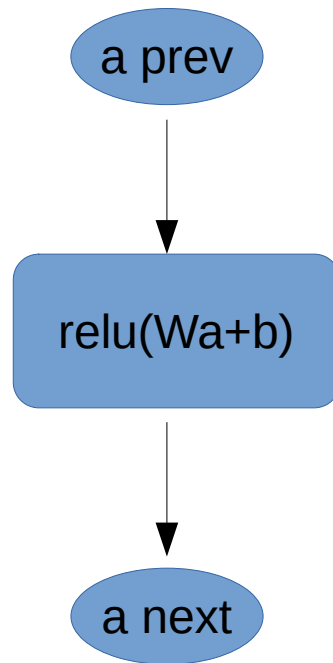


$$f_{w,b}(x) = \text{relu}(W \cdot a + b) + X$$

Architectures: residual network

Idea: let's create a shortcut for gradients

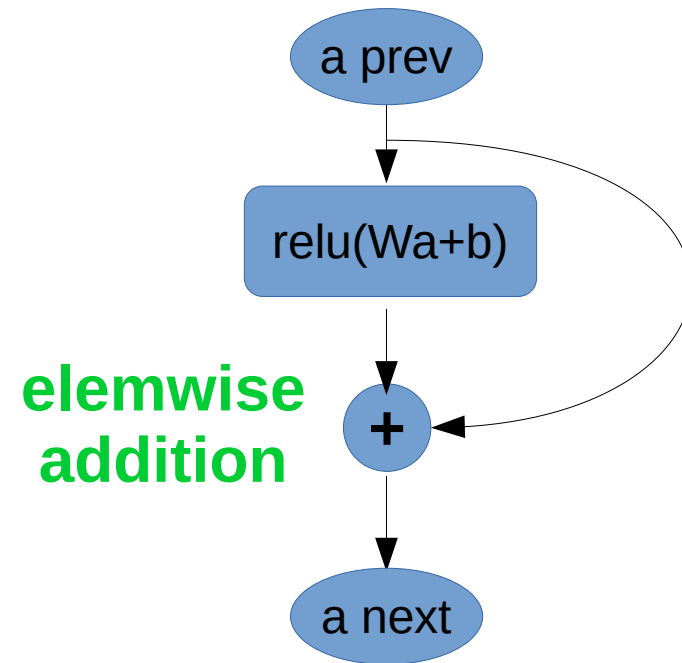
Normal layer



$$f_{w,b}(x) = \text{relu}(W \cdot a + b)$$

$$\nabla f_{w,b}(x) = \nabla \text{relu}(W \cdot a + b)$$

Residual layer



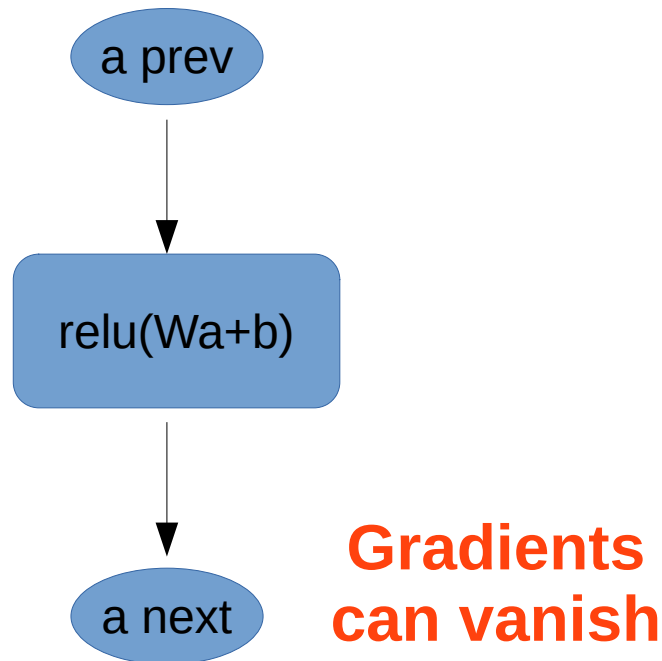
$$f_{w,b}(x) = \text{relu}(W \cdot a + b) + X$$

???

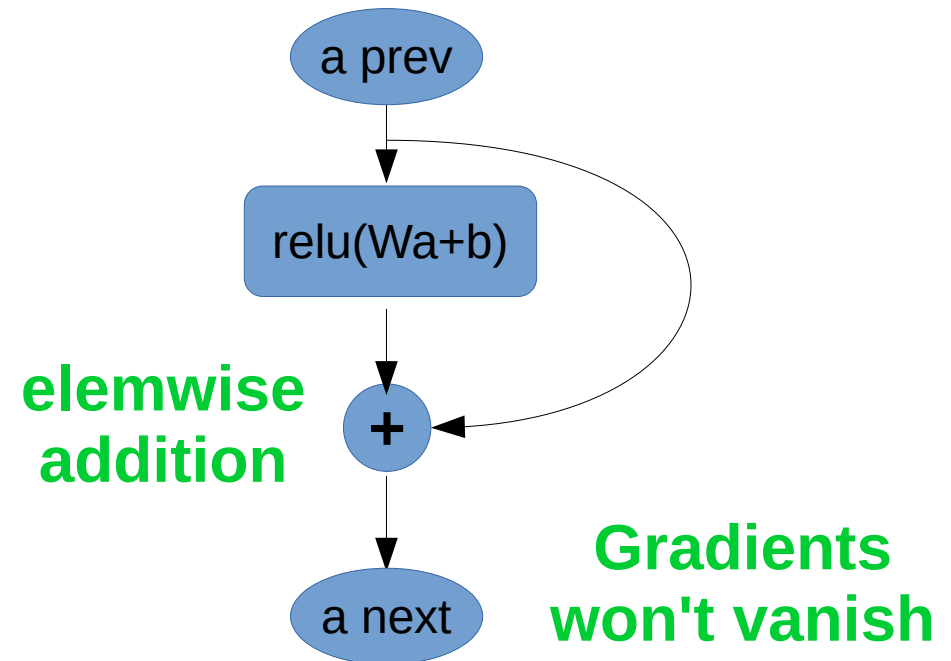
Architectures: residual network

Idea: let's create a shortcut for gradients

Normal layer



Residual layer



$$f_{w,b}(x) = \text{relu}(W \cdot a + b)$$

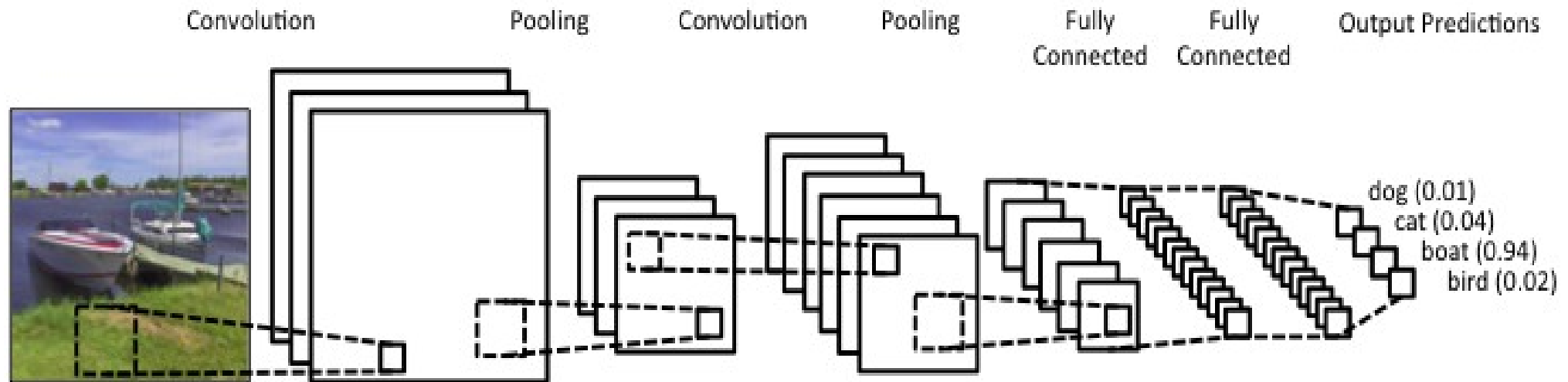
$$\nabla f_{w,b}(x) = \underline{\nabla \text{relu}(W \cdot a + b)}$$

$$f_{w,b}(x) = \text{relu}(W \cdot a + b) + X$$

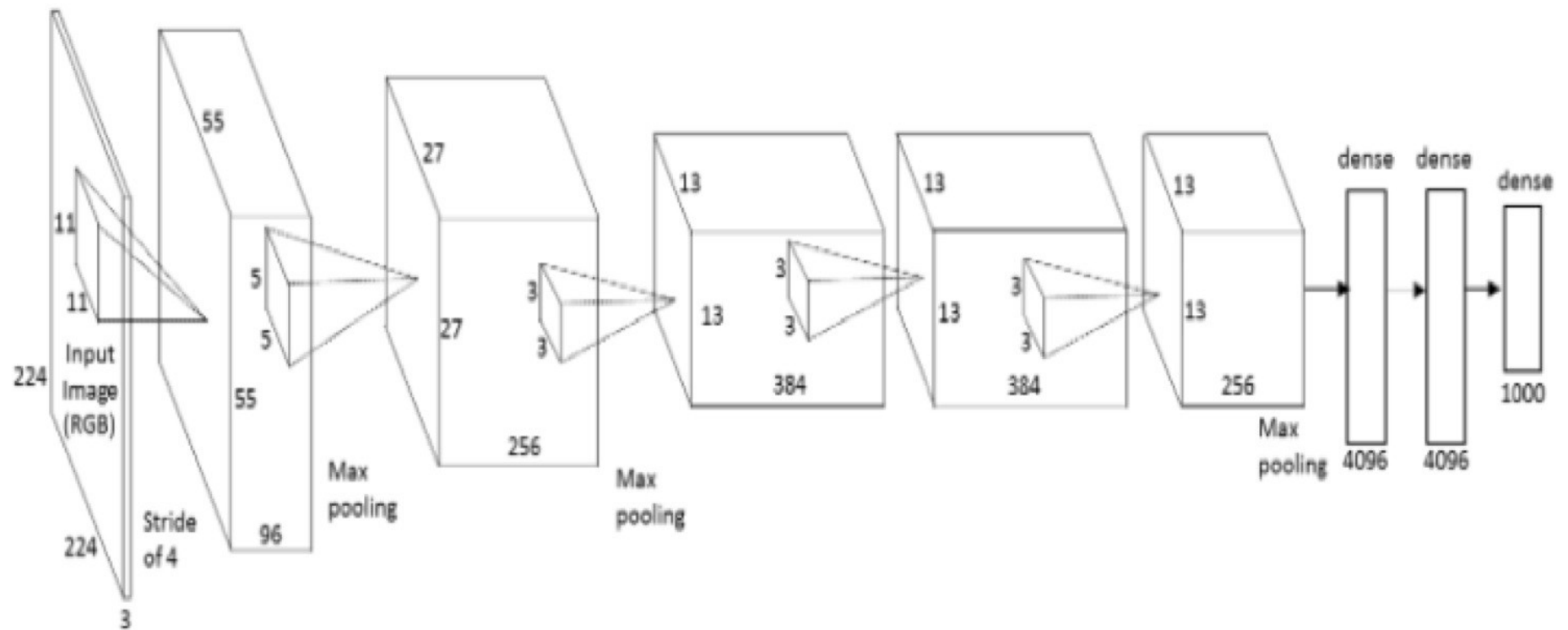
$$\nabla f_{w,b}(x) = \underline{\nabla \text{relu}(W \cdot a + b)} + \underline{\vec{1}}$$

BACK TO COMPUTER VISION

Previously...

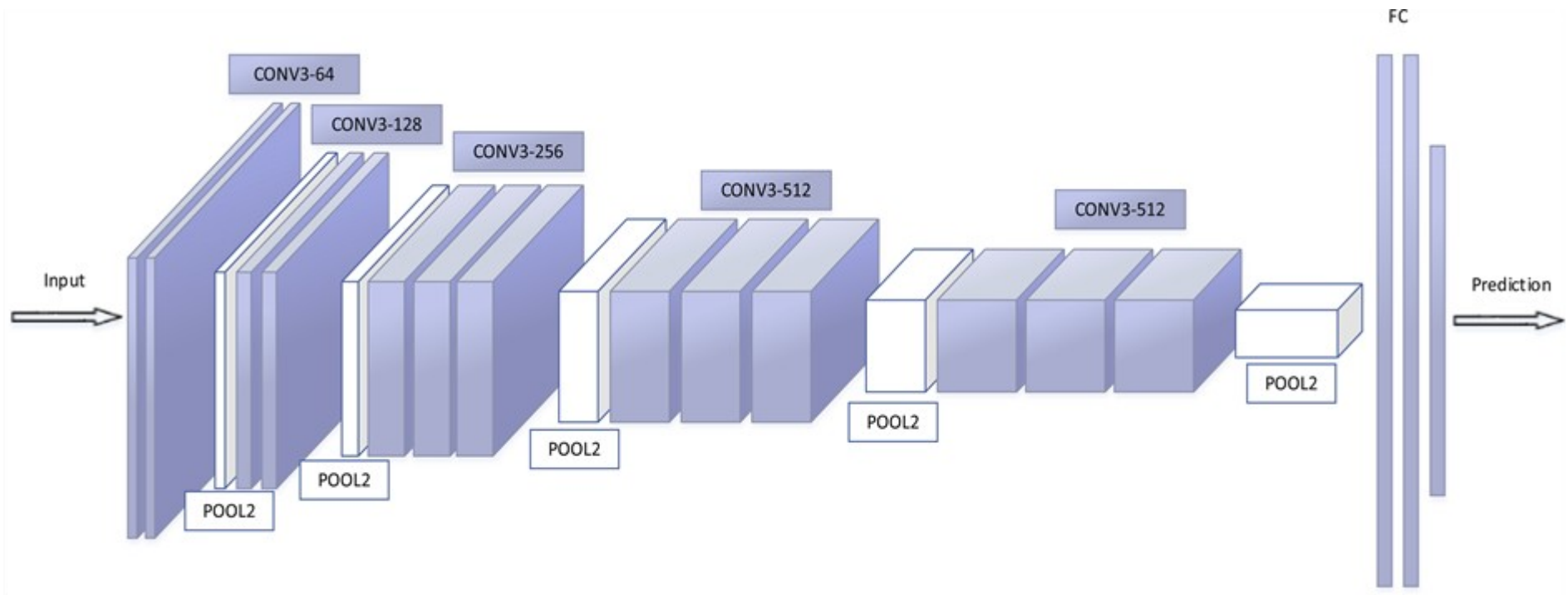


Alexnet



(Krizhevsky et al.)

VGG16



Karen Simonyan and Andrew Zisserman

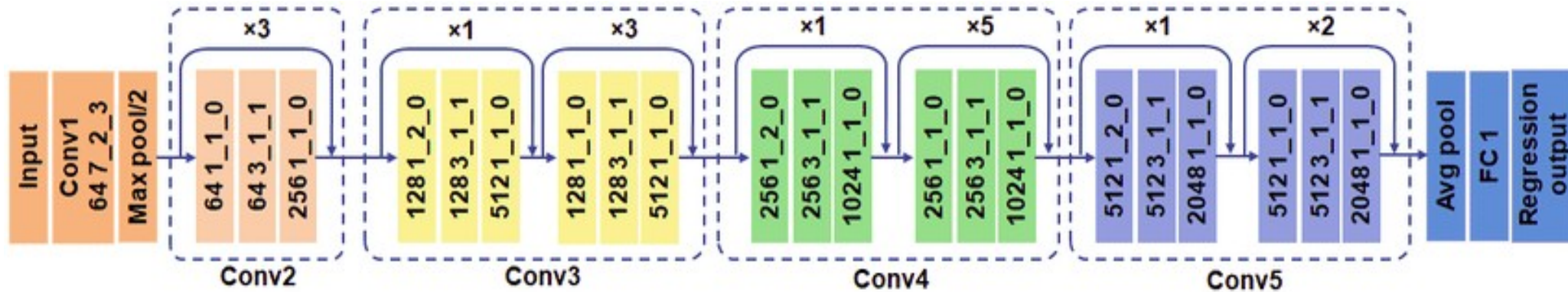
Problem with large networks

What you sign for if you stack 1000 layers:

- `MemoryError(0x...)`
- Gradients can vanish
- Gradients can explode
- Activations can vanish
- Activations can explode
- Overfits like crazy

ResNet & DenseNet

ResNet: add up activations



<https://arxiv.org/abs/1512.03385>

DenseNet: concatenate activations

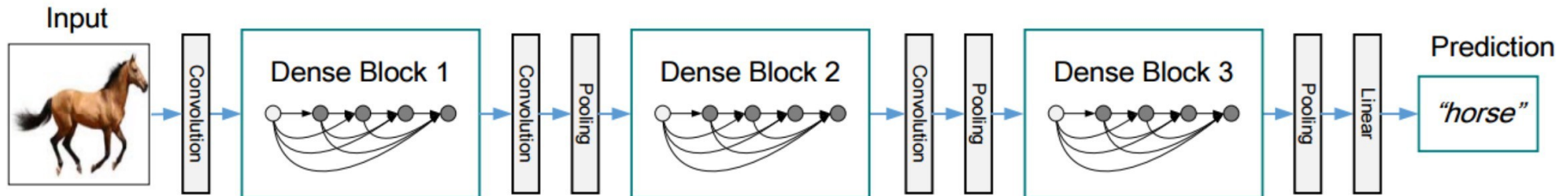
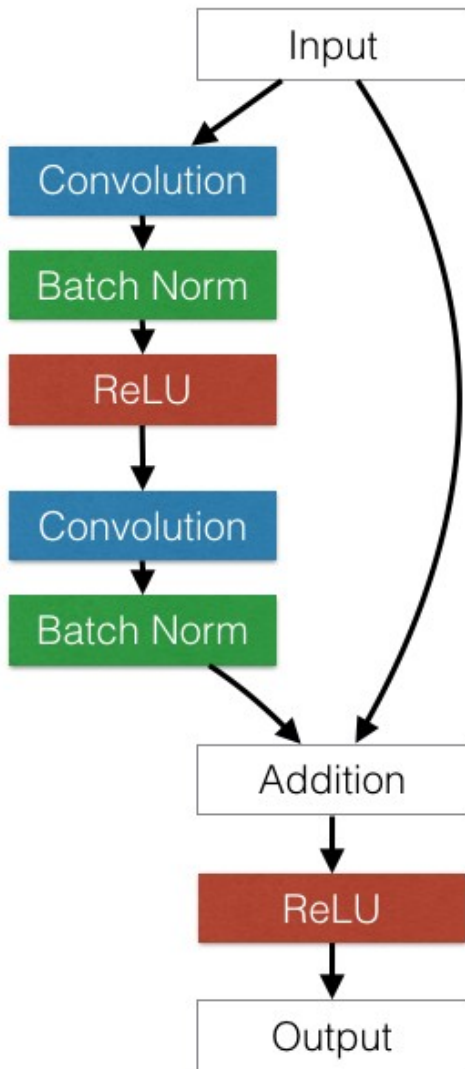


Figure 2. A deep DenseNet with three dense blocks. The layers between two adjacent blocks are referred to as transition layers and change feature map sizes via convolution and pooling.

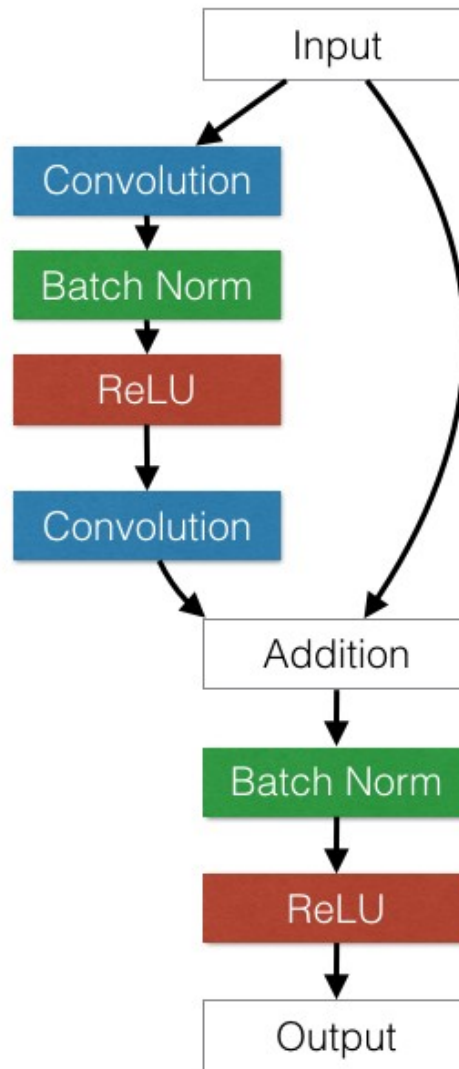
<https://arxiv.org/abs/1608.06993>

ResNet architectures

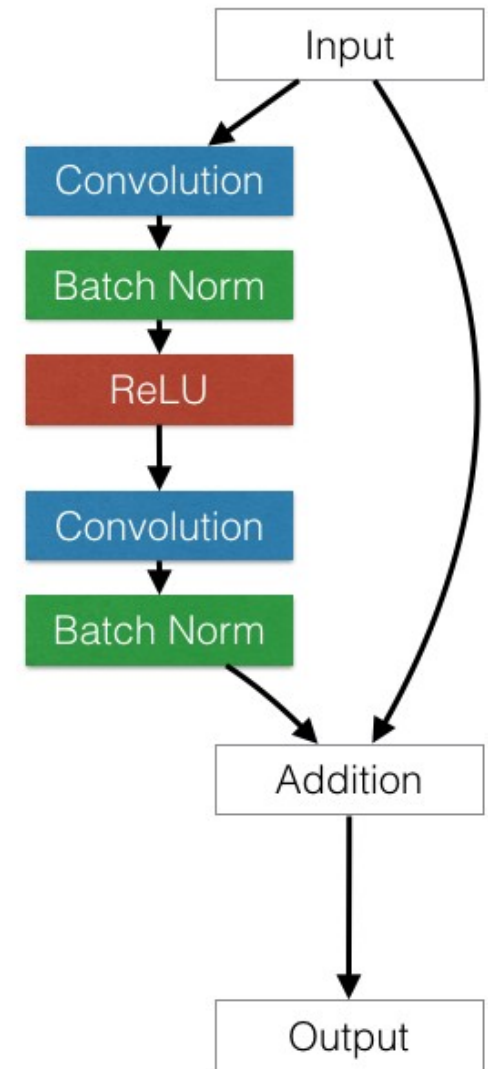
Reference paper



Batch Norm after add



No ReLU

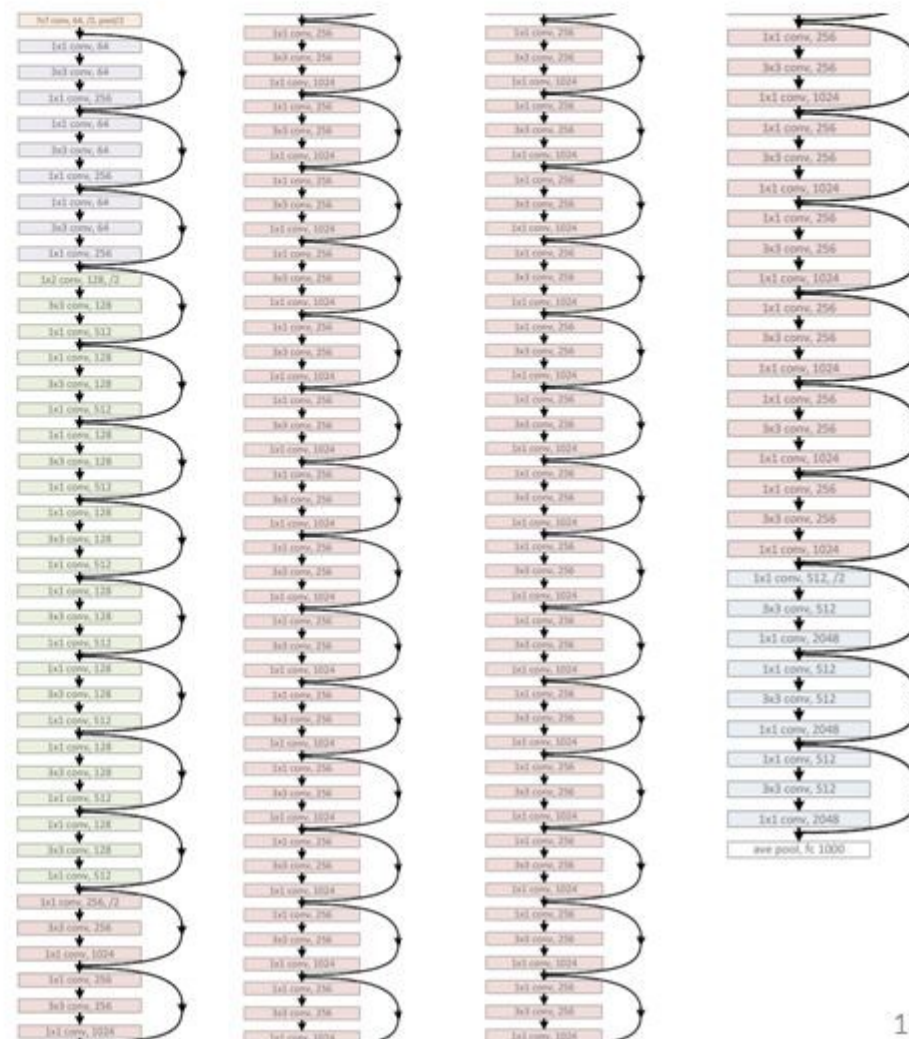


Resnet-152

Each layer learns to “modify”
previous activation

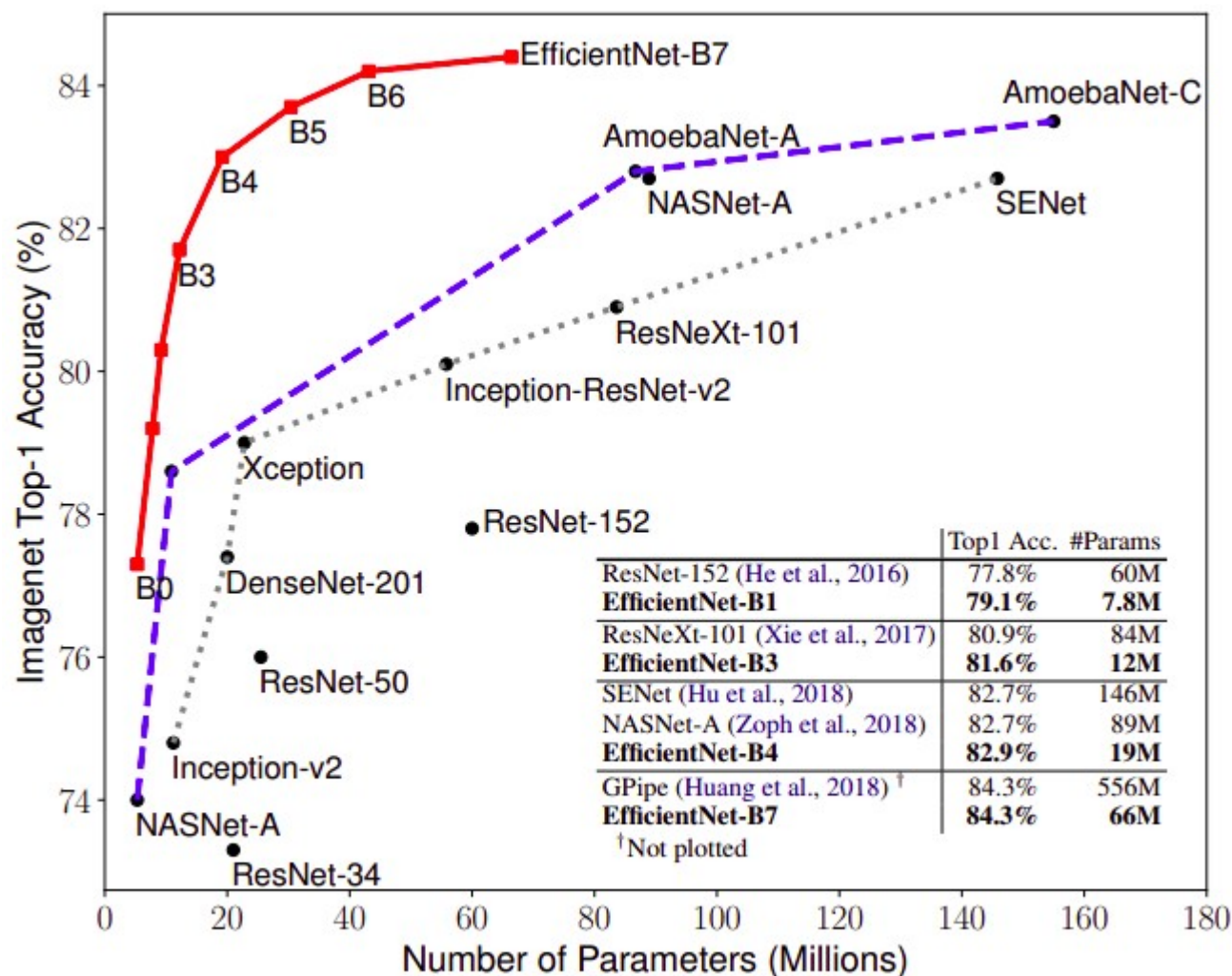
Much less parameters than
vgg16/vgg19

Can be used for other tasks:
segmentation, detection,
captioning
(just like vgg/inception/...)



Modern approach: EfficientNet

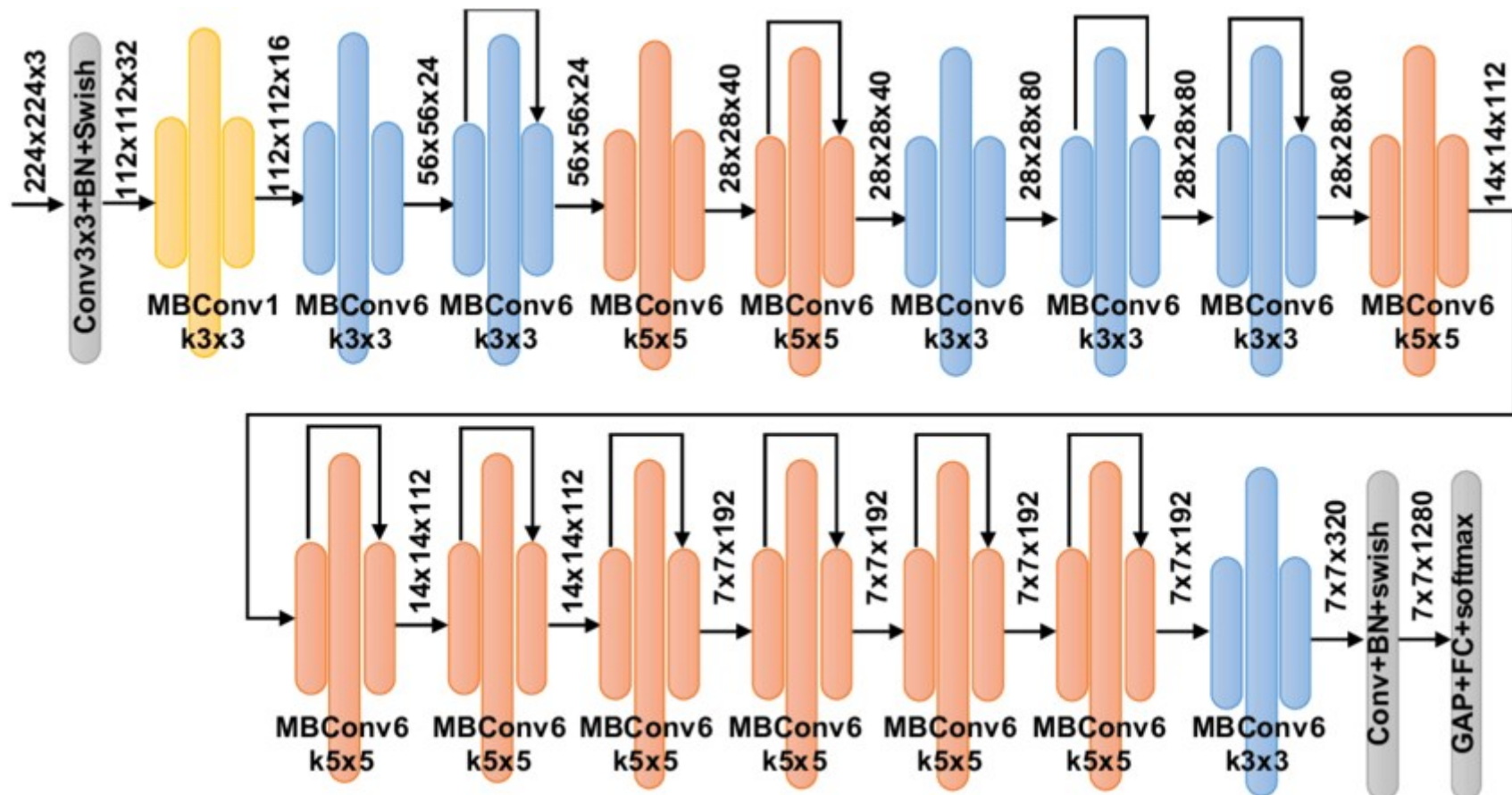
<https://arxiv.org/abs/1905.11946>



Modern approach: EfficientNet

<https://arxiv.org/abs/1905.11946>

- Designed base network with neural architecture search (not the first paper to do so)



Modern approach: EfficientNet

<https://arxiv.org/abs/1905.11946>

- Found optimal scaling ratio for depth/width/res

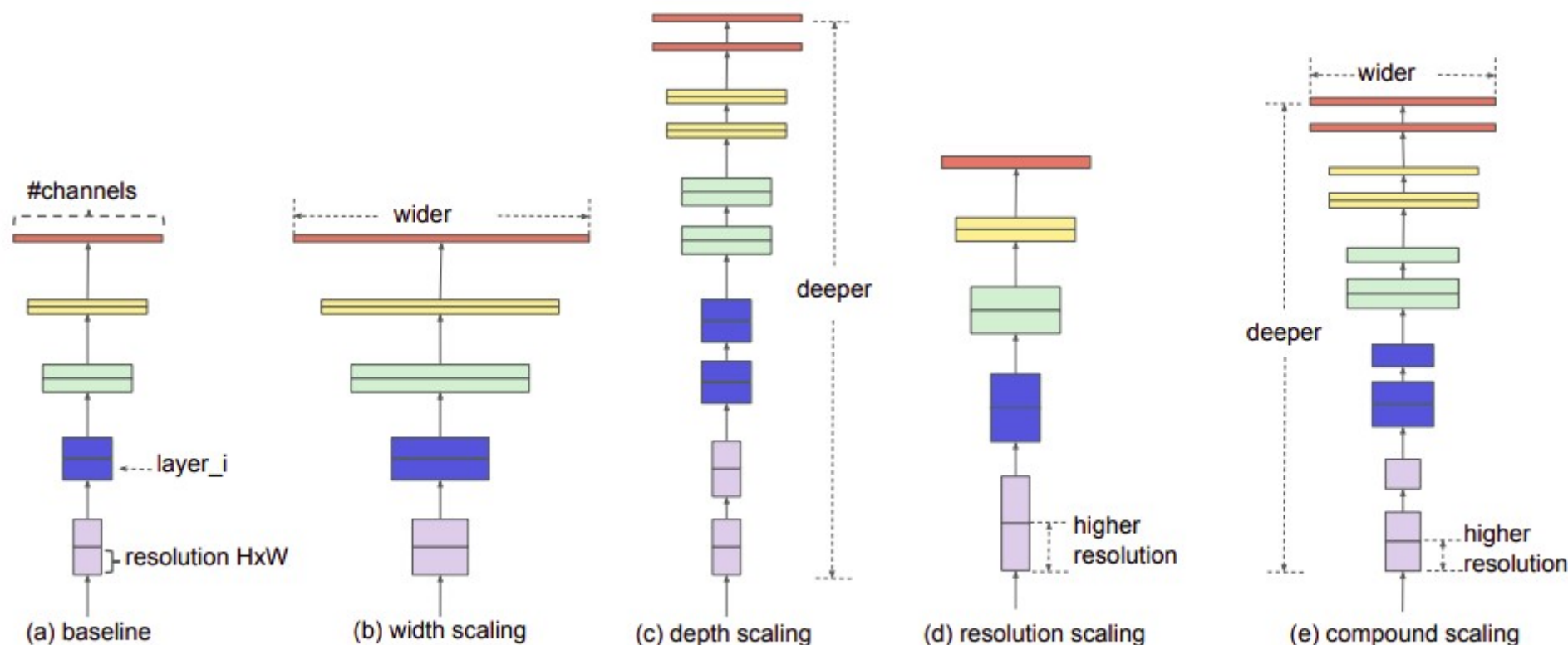


Figure 2. Model Scaling. (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.

Modern approach: EfficientNet

<https://arxiv.org/abs/2104.00298>

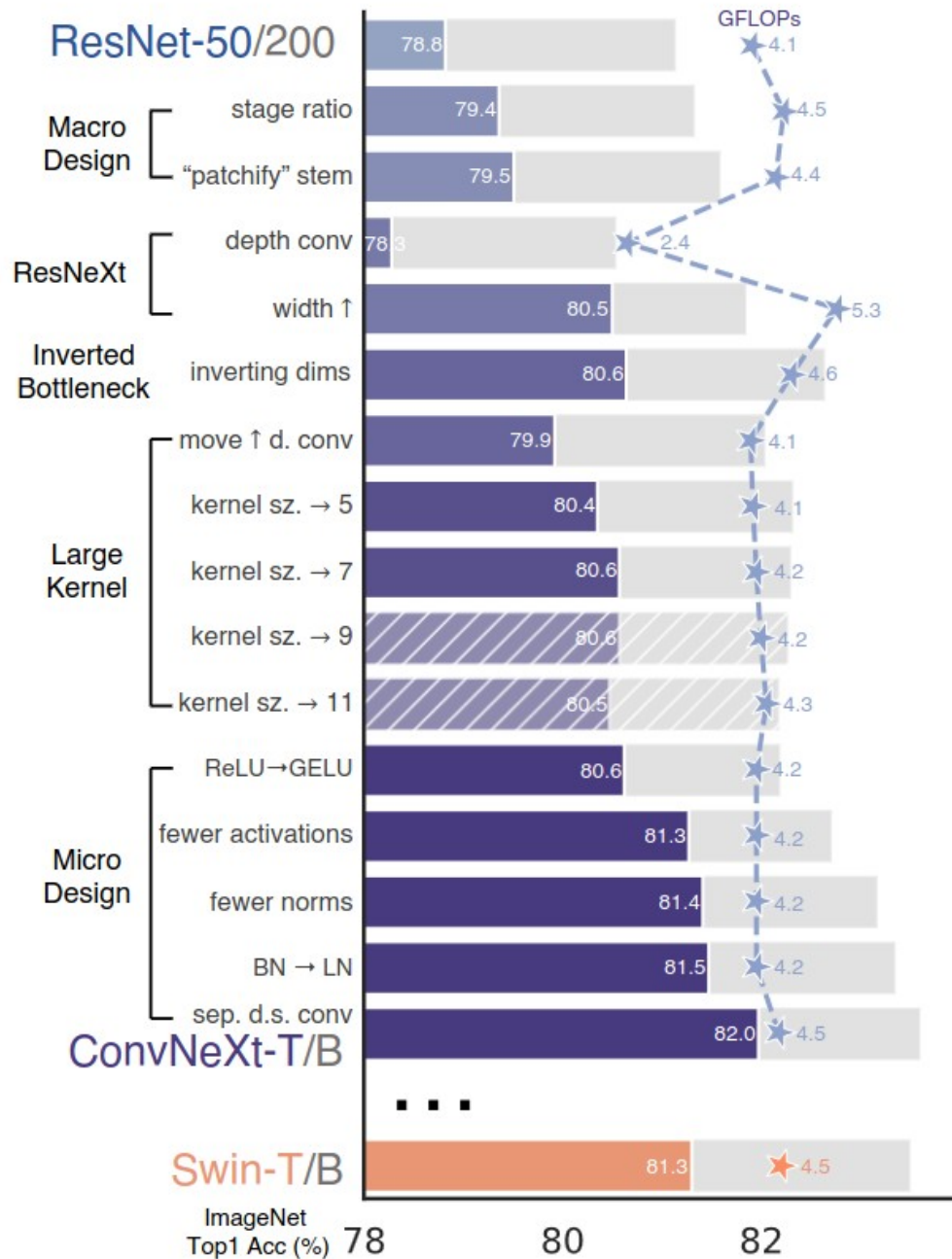
- Training faster with curriculum learning



Figure 4. Training process in our improved progressive learning – It starts with small image size and weak regularization (epoch=1), and then gradually increase the learning difficulty with larger image sizes and stronger regularization: larger dropout rate, RandAugment magnitude, and mixup ratio (e.g., epoch=300).

Modern approach: ConvNext

<https://arxiv.org/abs/2201.03545>



1. gather architecture insights and training tricks from post-resnet papers
2. apply them all to resnet and analyze which work best
3. ???
4. SOTA!!!

Modern stuff

- Architectures:

EfficientNetV2 <https://arxiv.org/abs/2104.00298>

CNN for 2020 <https://arxiv.org/abs/2201.03545>

Vision transformers - later

- Interpretability

<https://distill.pub/2018/building-blocks>

Note: data augmentation



- Idea: we can get N times more data by tweaking images.
- If you rotate cat image by 15° , it's still a cat
- Rotate, crop, zoom, flip horizontally, add noise, etc.
- Sound data: add background noises

Nuff

Let's train some CNNs!

